Computer Vision / CNNs and Biases in Machine Learning

Nov. 11, 2025 - Lecture 8

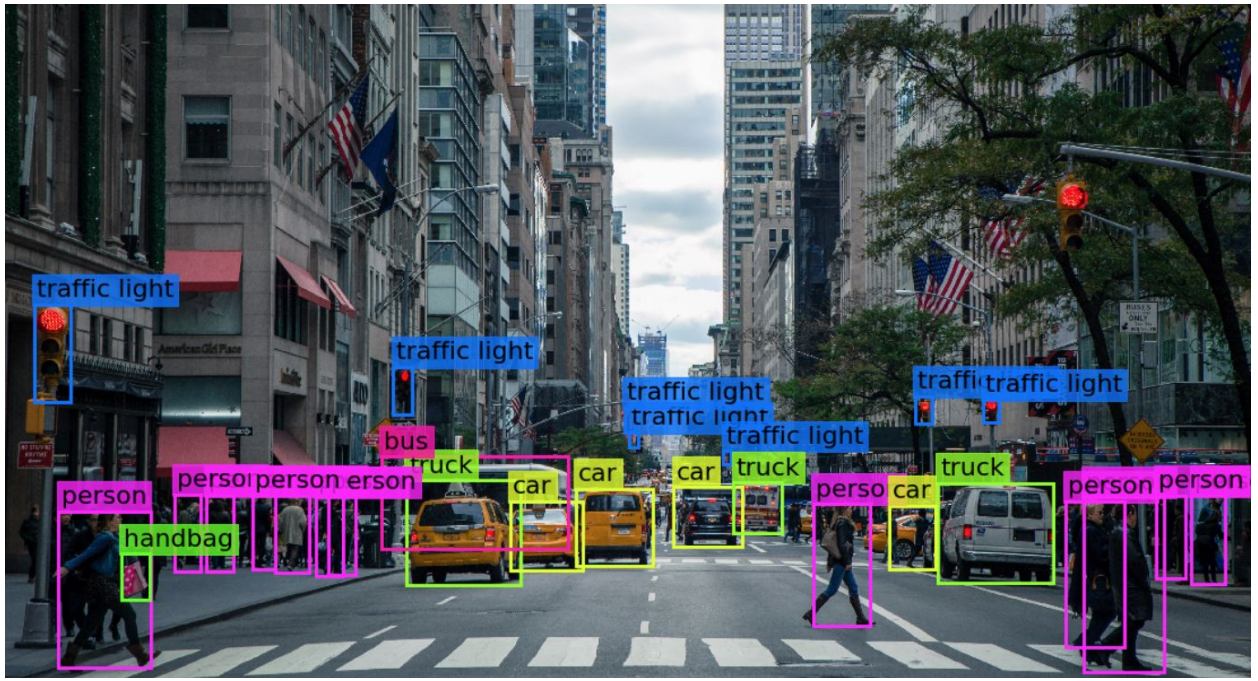
Outline

- Computer Vision
- Representing images
- CNNs
  - Convolution, filters, stride, pooling, padding, image augmentation
- Other CNNs and transfer learning
- Bias, fairness, mitigating bias and improving fairness

Computer Vision

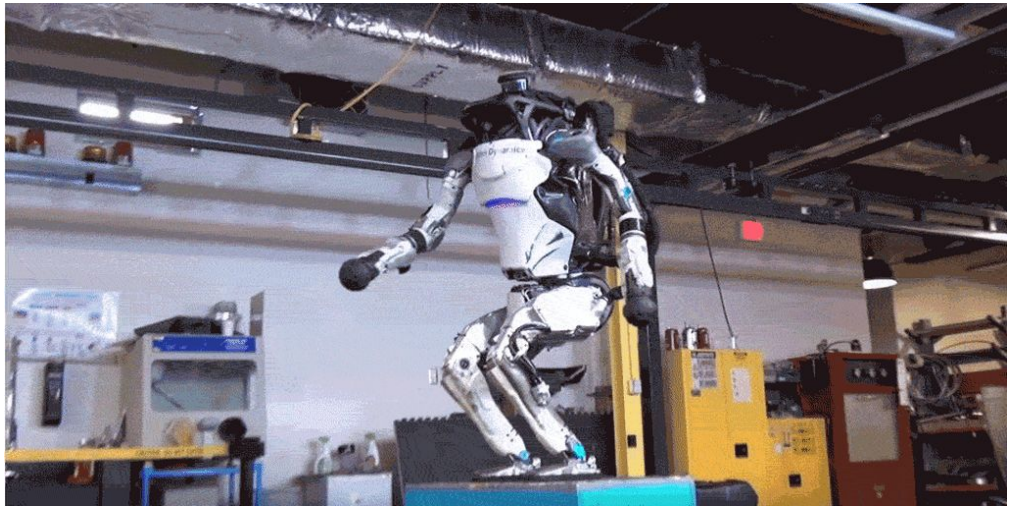
Computer vision is a field of artificial intelligence to automate tasks that the **human visual system** can do.

Computer vision is concerned with **extracting information** from digital images, videos and other visual inputs, and **taking action** based on those inputs (e.g. self-driving cars, medical imaging, object detection).



Computer Vision: Some Applications

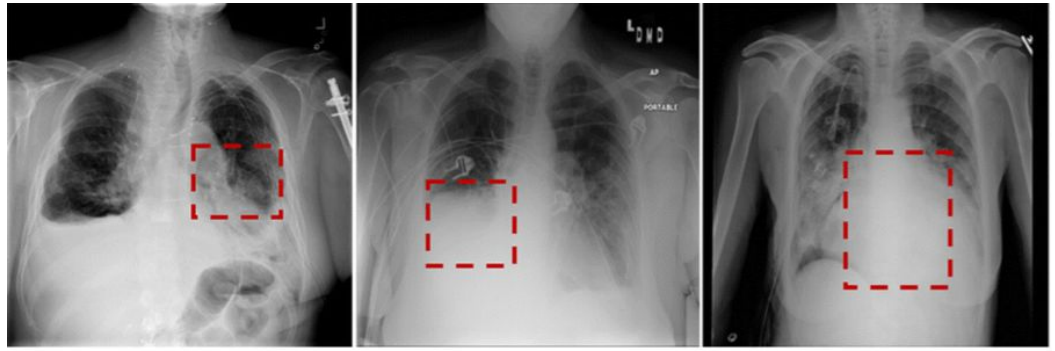
Robotics



Autonomous Driving

Mobile Computing

Biology and Medicine



How do we Represent Images?

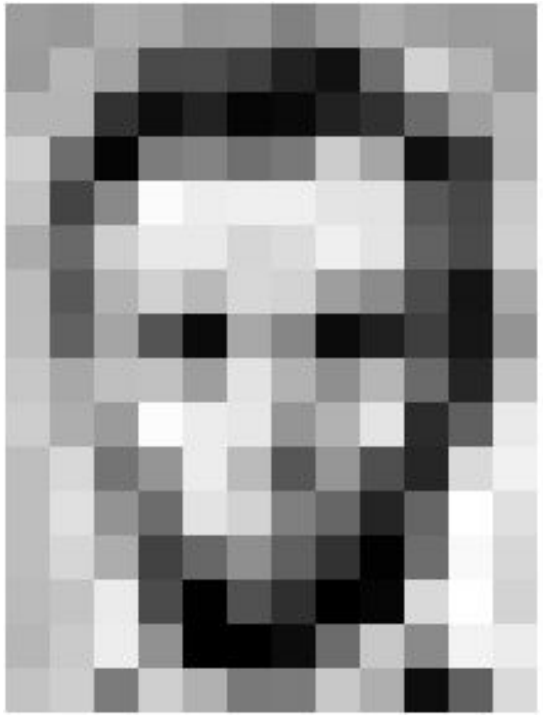
Images are numbers!

A **grayscale image** can be represented as a **2D array** of pixel values, where each value represents the brightness of a pixel.

A **color image** is just a "stack" of 3 grayscale images: one for **red** values, one for **green** values, and one for **blue** values.

Grayscale Images

A **grayscale image** can be represented as a **2D array** of pixel values, where each value represents the brightness of a pixel.

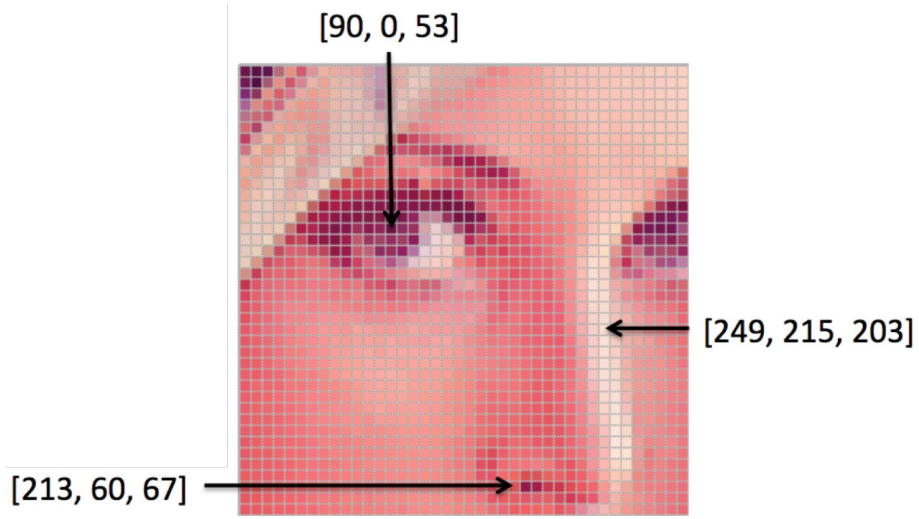


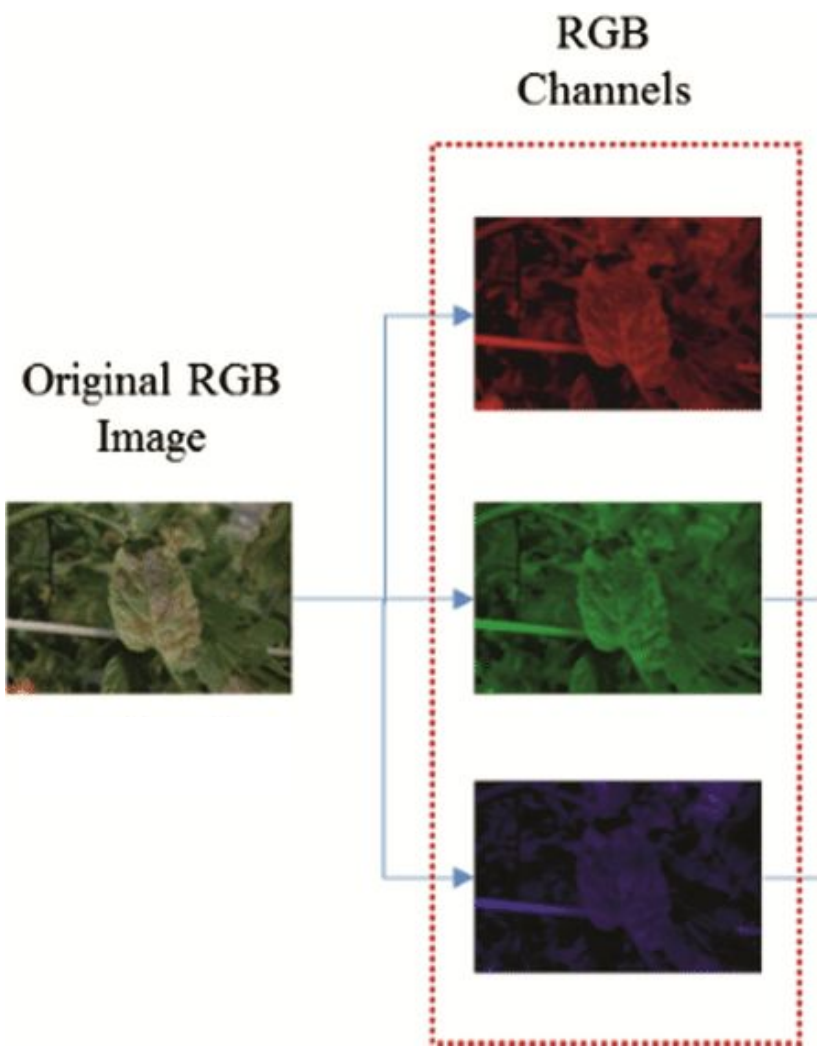
157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

Color Images

A **color image** can be represented as a 3D array of red, green, and blue values.





Why Not NNs*?

1. NNs are **fully connected** and have too many weights to train - does not scale well.

2. **Small changes** in the image can lead to poor performance.

3. Treats each pixel independently and **ignores the spatial structure** of images.

*Note: we are talking about **feed-forward neural networks** here.

Why Not NNs?

1. NNs are **fully connected** and have too many weights to train - does not scale well.

2. **Small changes** in the image can lead to poor performance.

3. Treats each pixel independently and **ignores the spatial structure** of images.

CNNs

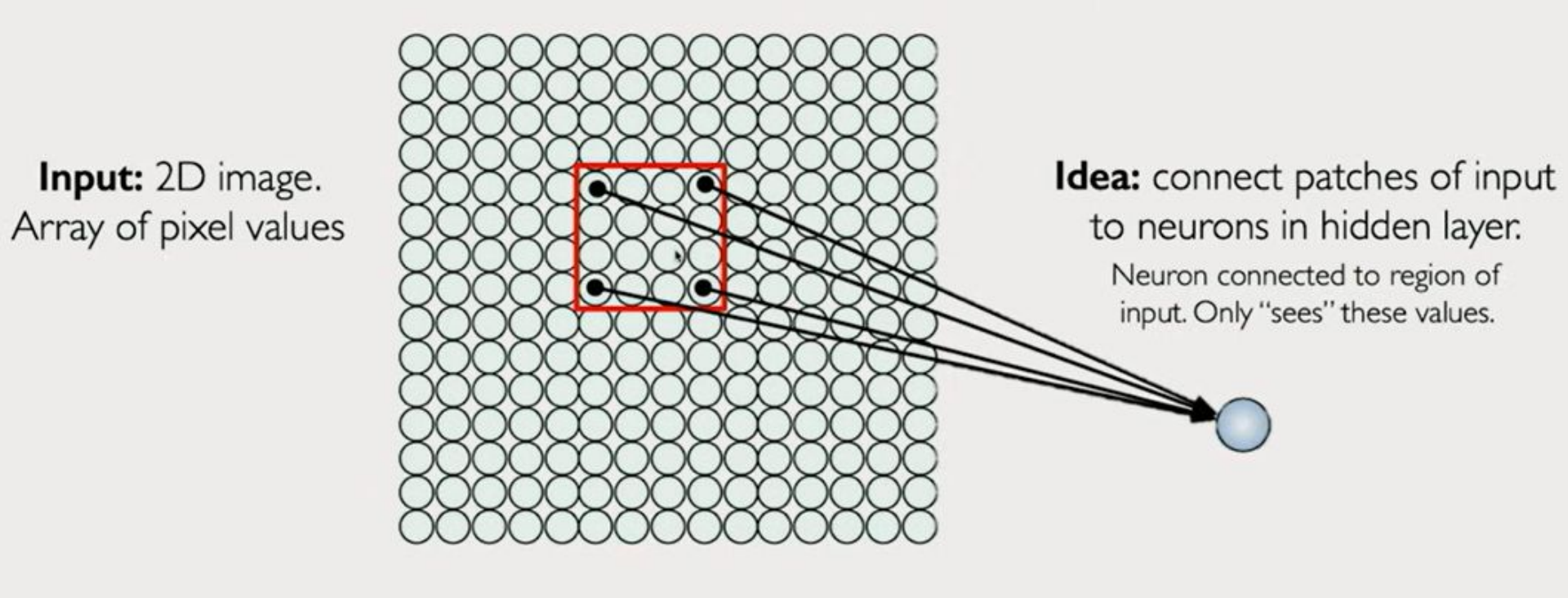
Look at small parts of the image at a time, are **locally connected.**

Tolerates small shifts in pixel location!

Takes into account the spatial structure.

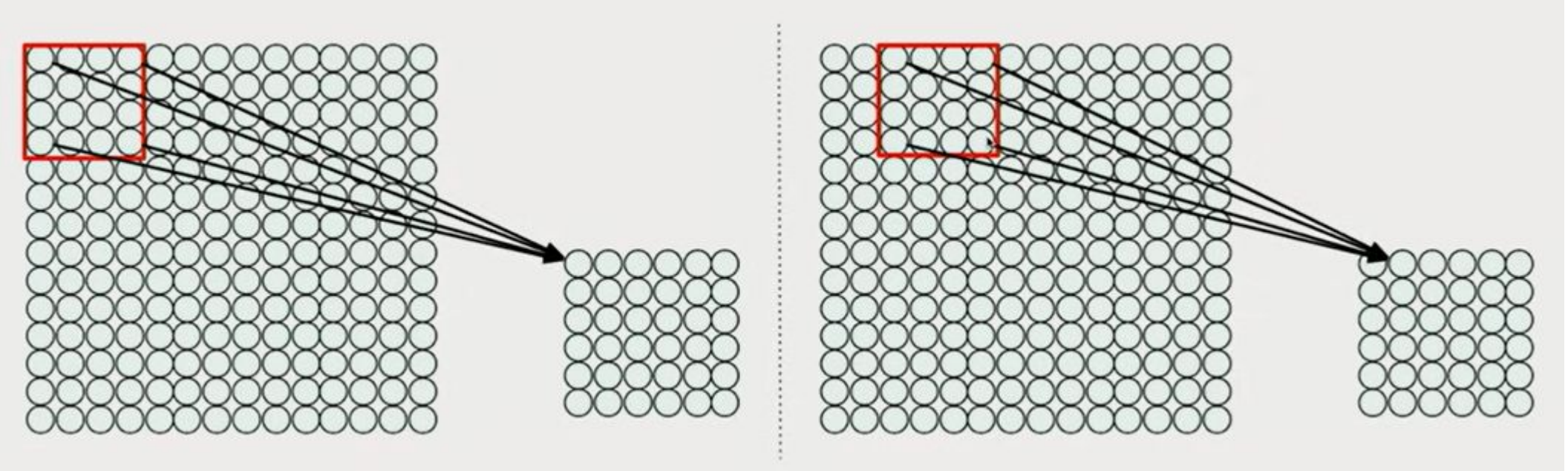
CNNs are Locally Connected

In order to **extract spatial information** from image data, CNNs are **not fully connected.** Instead, they only maintain some connections.



CNNs are Locally Connected

Each **patch** in one layer corresponds to **one neuron** in the next layer.



Convolutional Neural Networks (CNNs)

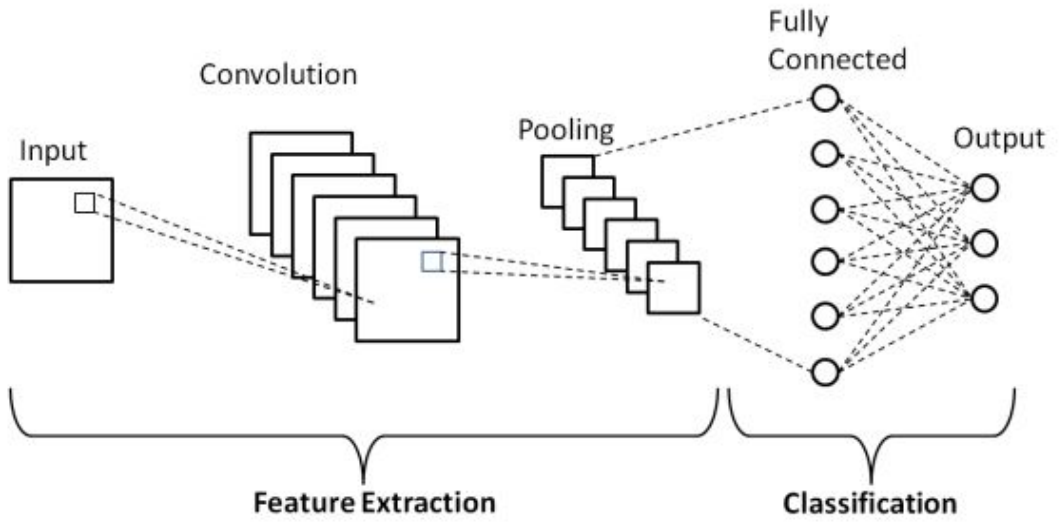
Convolutional neural networks (CNNs) are a type of neural network designed to **work with visual data.**

- Before CNNs, manual, time-consuming feature extraction methods were used to identify objects in images. **CNNs can do this automatically.**

Convolutional Neural Networks (CNNs)

CNNs have a different structure from feed forward neural networks.
CNNs have 3 main types of layers:

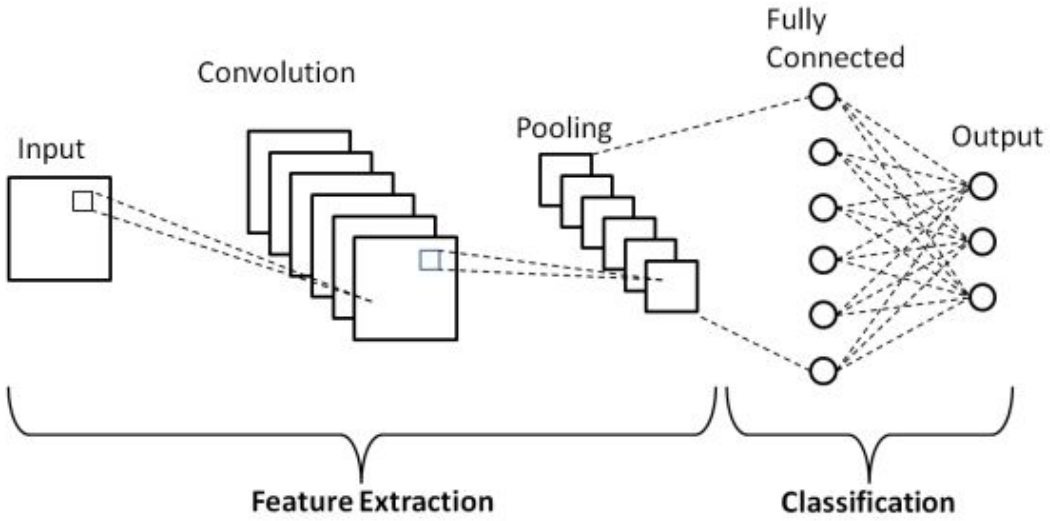
1. **Convolutional layer**
2. **Pooling layer**
3. **Fully-connected (FC) layer**



Convolutional Neural Networks (CNNs)

With each layer, the CNN **increases in its complexity.**

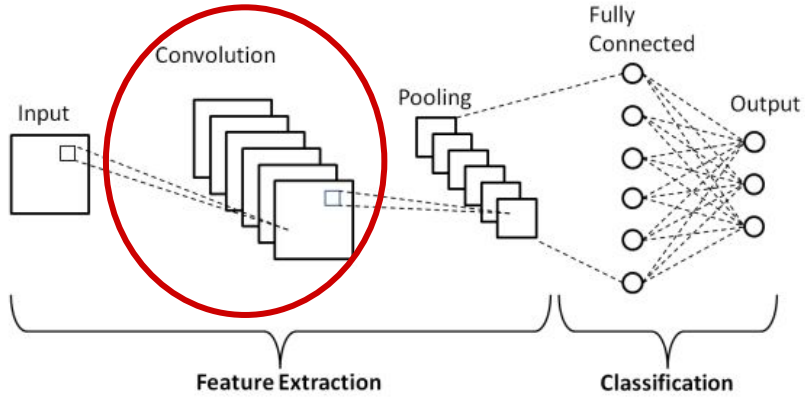
- Earlier layers focus on **simple features** (e.g. colors and edges)
- As the image data progresses through the layers of the CNN, it starts to recognize larger elements or shapes of the object until it finally identifies the intended object.



1. Convolutional Layers

Convolutional layers are responsible for finding **patterns or features** in the image, such as edges, corners, or textures.

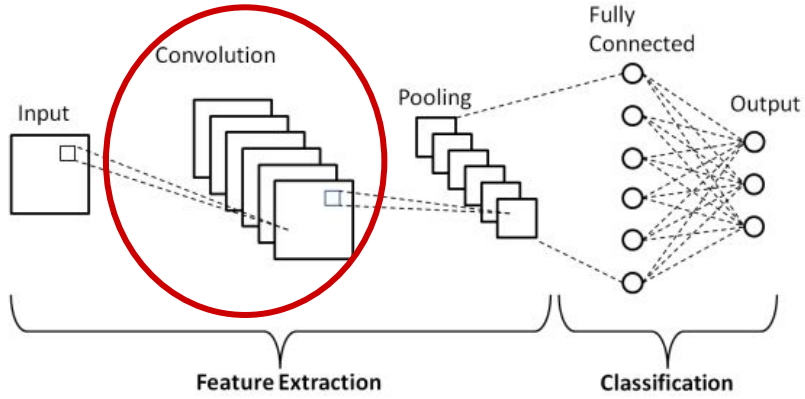
- They use **filters / kernels** that slide over the image, performing a mathematical operation called **convolution.**
- The output is a new array called a **feature map**, which shows where in the original image those patterns were found.



1. Convolutional Layers

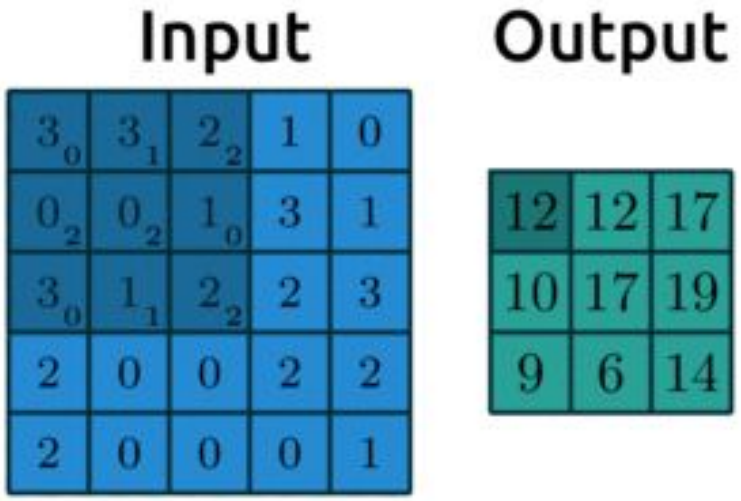
Convolutional layers are responsible for finding **patterns or features** in the image, such as edges, corners, or textures.

- They use **filters / kernels** that slide over the image, performing a mathematical operation called **convolution.**



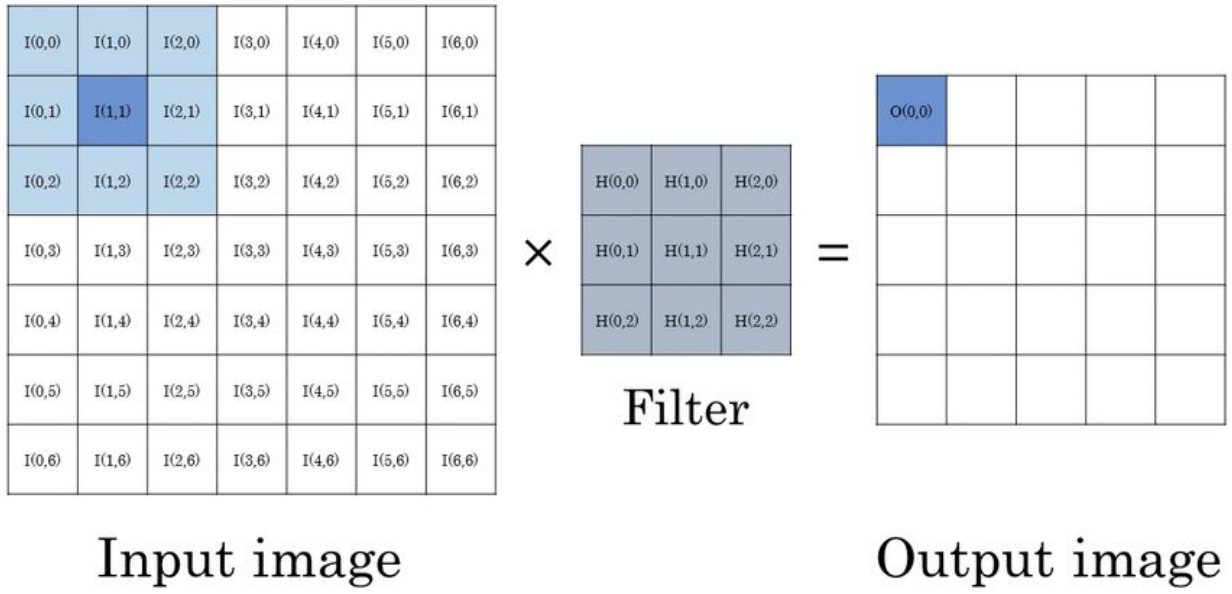
Convolution

Convolution the process of moving a filter across an image, checking if the feature is present.



Filters / Kernels

A **filter / kernel** is a small matrix used to apply some effect to an image (e.g. blurring, Sobel, sharpen). To apply a filter, we use **convolution.**



How do we Apply Filters?

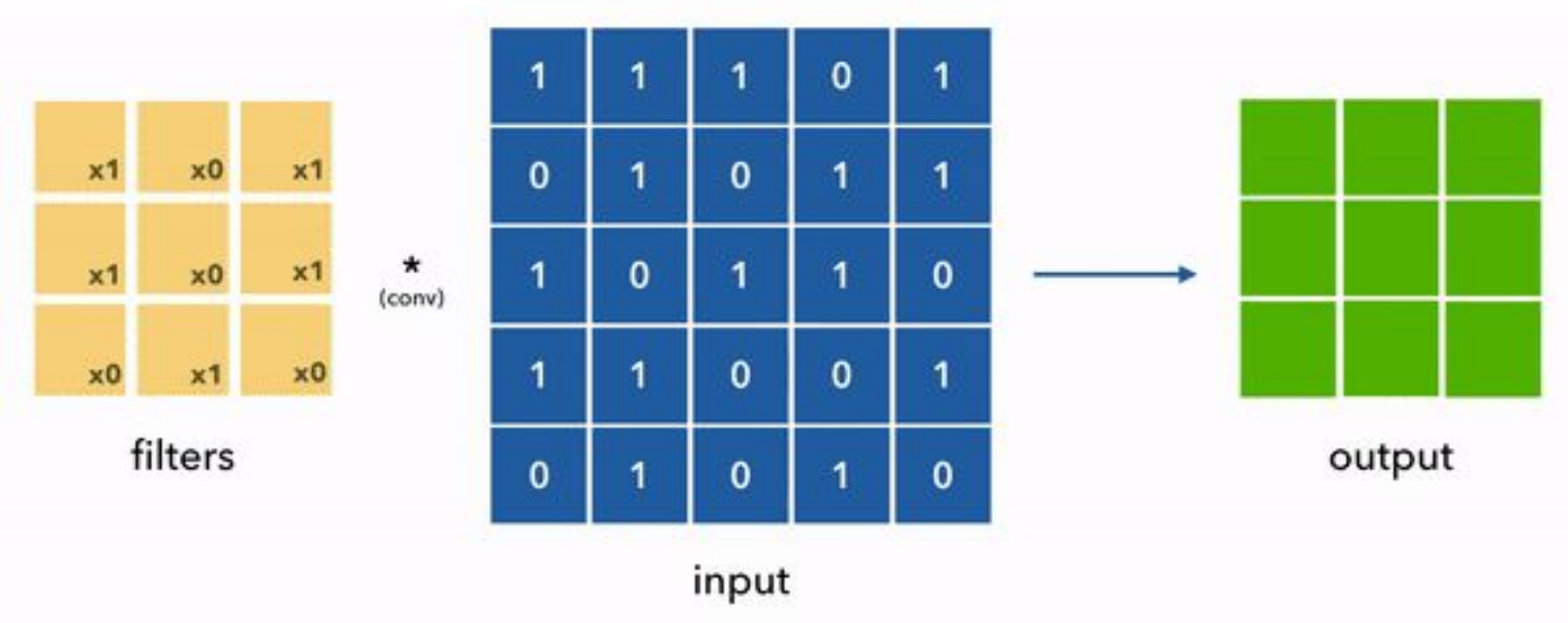
1. **Overlay** the filter on the image.
2. Compute the **dot product** (Multiply together each overlapping pixel value).
3. Sum up these values.

Repeat as you slide the filter over the image. The resulting values are stored in a **feature map.**

When we apply a filter to an input, we can say the filter is **convolved** with the input.

Filters / Kernels In Action

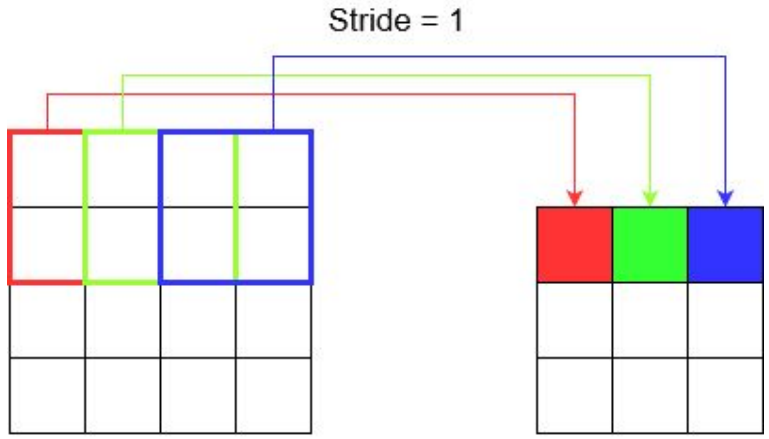
Convolution: applying a filter by sliding it over the input image, multiplying element-wise, and adding the outputs.

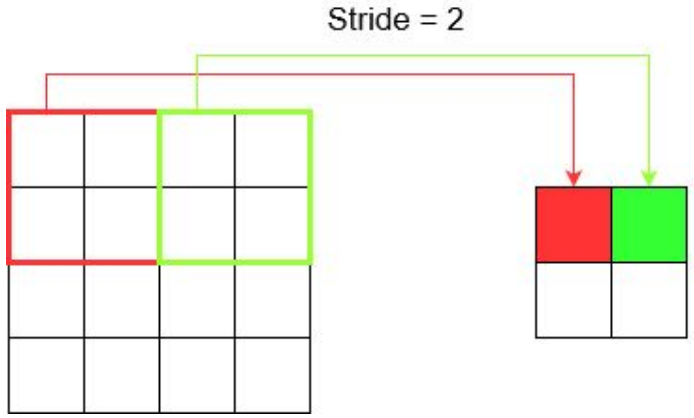


Stride

Stride: the number of pixels the kernel moves at each step.

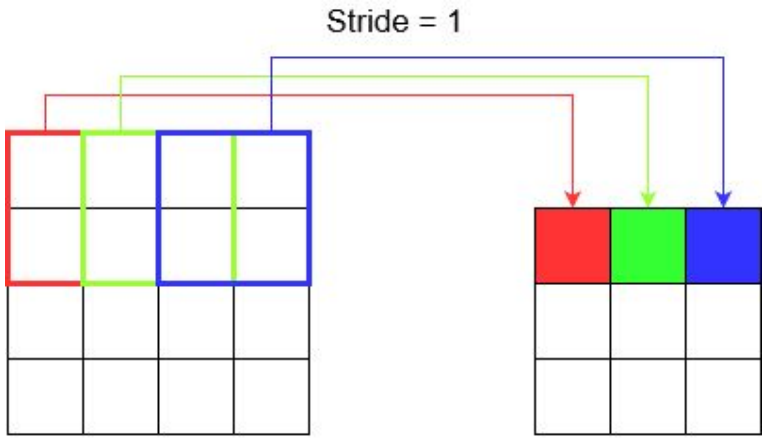
In the examples so far, we have used a stride of 1. Sometimes, we may want to use a larger stride.

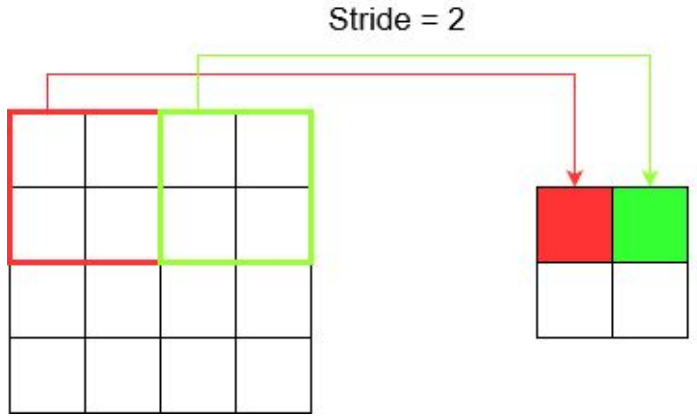




Stride

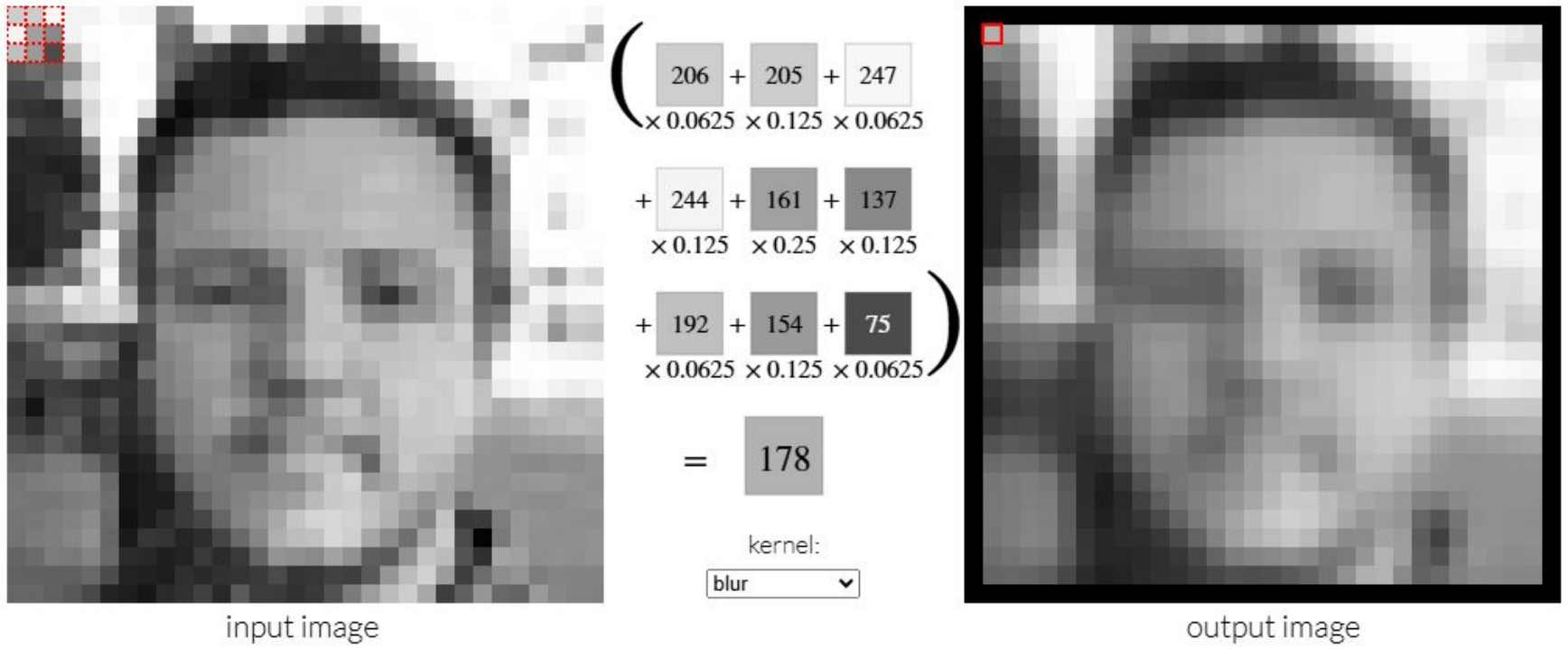
Larger stride = smaller output, fewer computational operations, more information loss





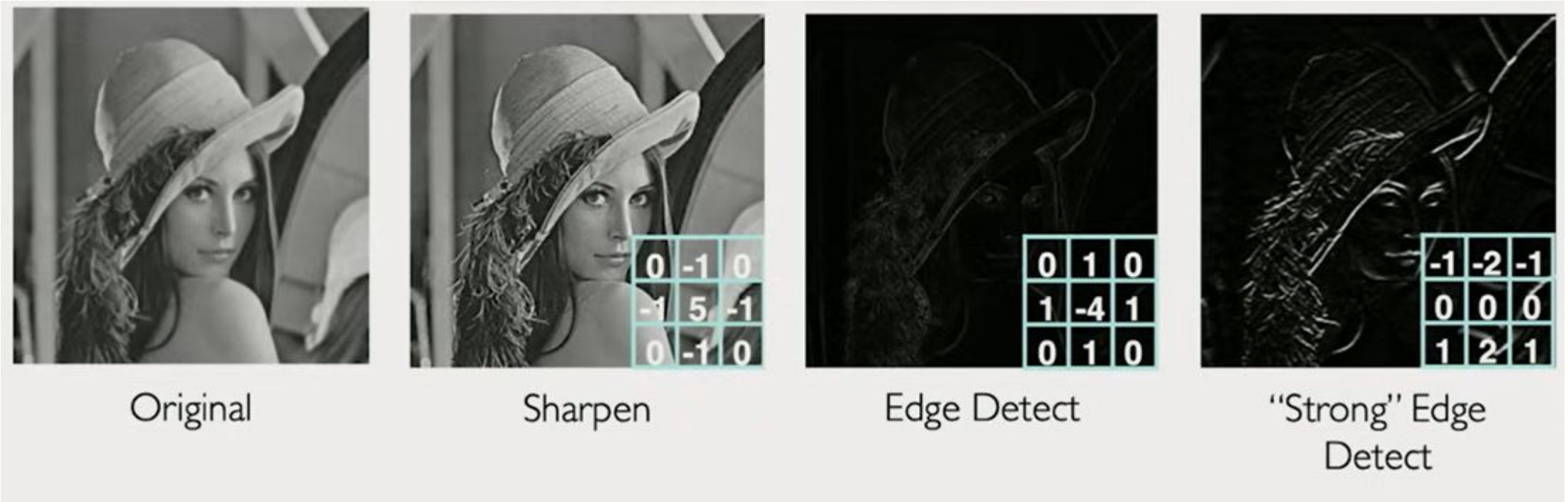
Filters / Kernels

There are **many types of filters** that produce all kinds of effects on images. For example, this image has a **blurring filter** applied to it.



Filters / Kernels

There are many **hand-engineered filters** already!



Filters / Kernels

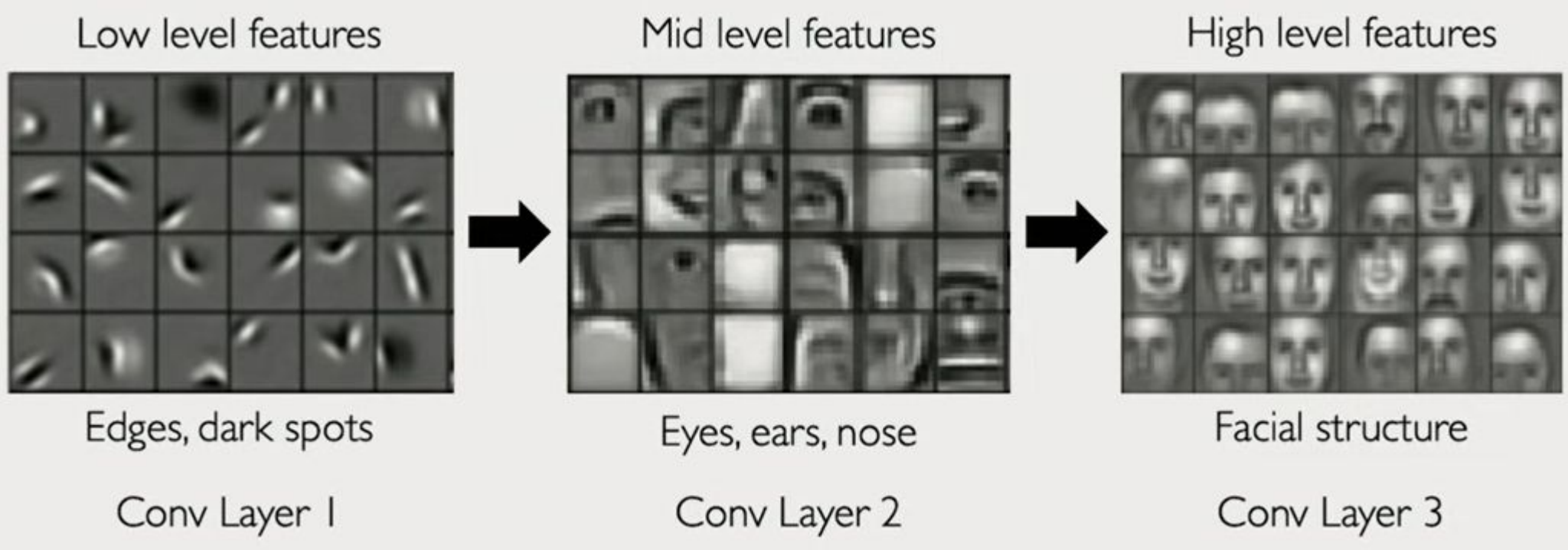
More commonly, **CNNs learn filters!**

- At the start of training, filters are initialized to **random numbers.**
- As the network trains, the numbers are adjusted to make the filters **detect patterns** that help with the task (e.g. recognizing shapes, objects, faces, etc.).

Early layers often learn **simple filters**, while deeper layers learn more **complex features.**

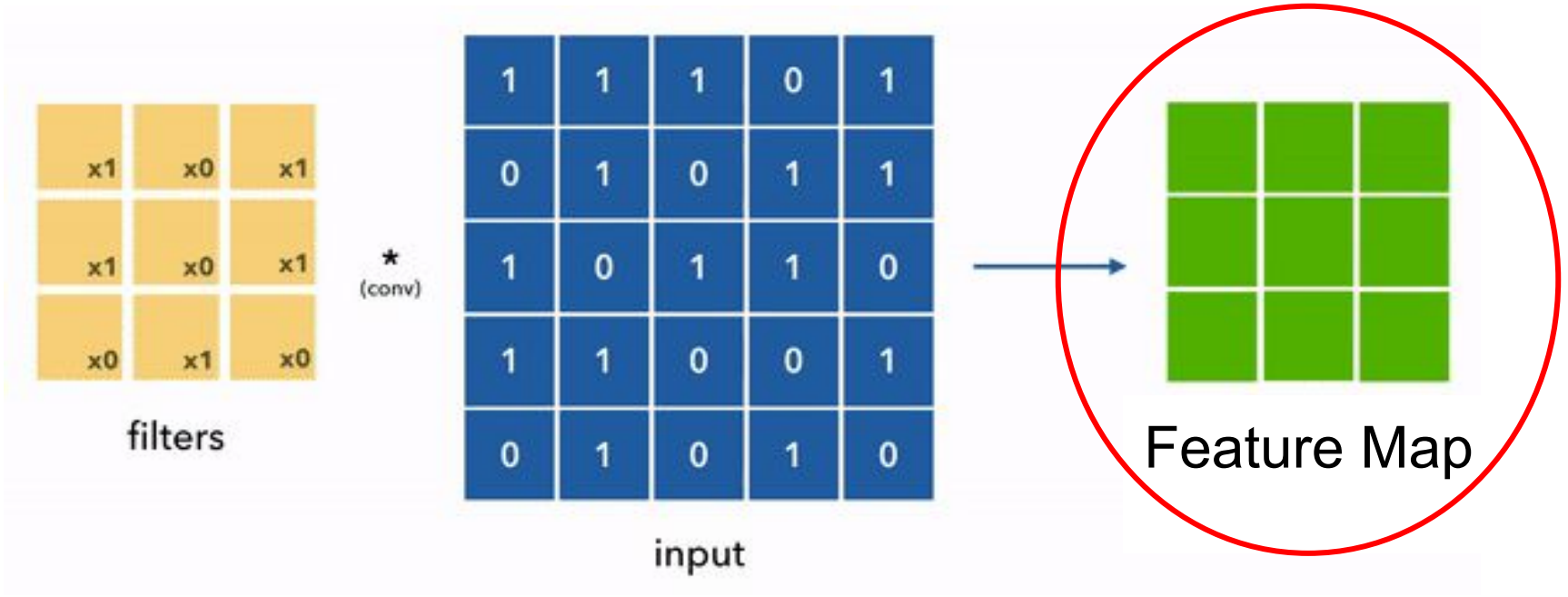
CNNs Learn Features Hierarchically

These images are **filters** from a CNN used to detect faces. None of these filters were hand-engineered, instead the CNN **learned** them!



Feature Maps

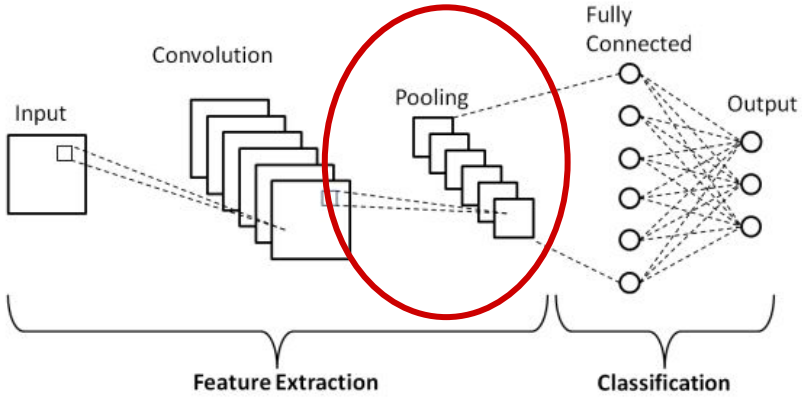
A **feature map** is the **output of a convolutional layer**. It is a matrix where each cell in it corresponds to a set of neighboring pixels.



2. Pooling Layers

Pooling layer (downsampling) conducts dimensionality reduction, reducing the size of the feature maps.

- Similar to the convolutional layer, the pooling operation sweeps a filter across the entire input, but the difference is that this filter **does not have any weights**.
- Instead, the kernel applies an **aggregation function (e.g. mean, max)** to the values within the receptive field, populating an output array.

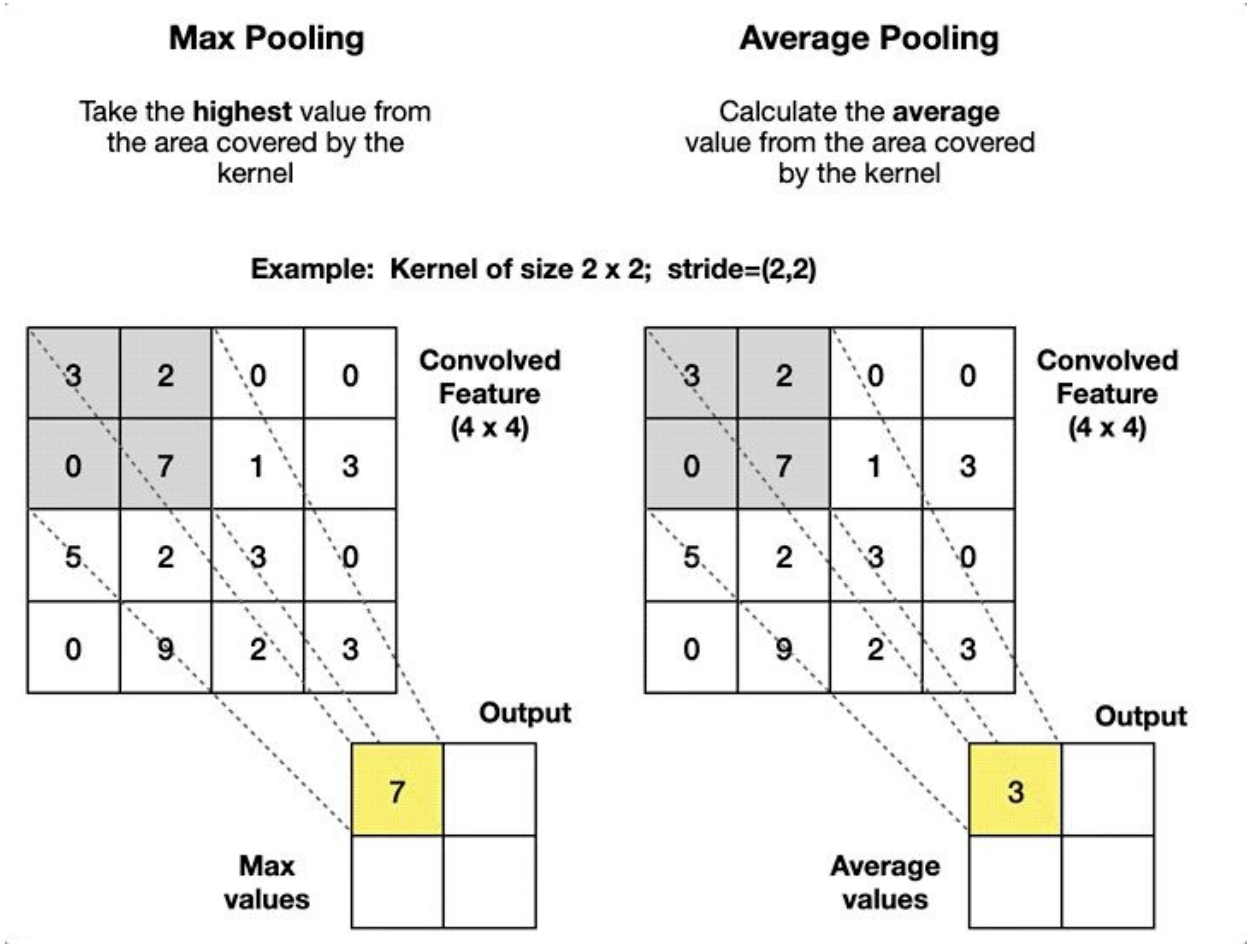


Pooling Layers

There are 2 main types of **pooling**:

1. **Max pooling:** As the filter moves across the input, selects the maximum value to output.
2. **Average pooling:** As the filter moves across the input, calculate the average value within the **receptive field** to send to the output array.

Pooling Layers



Pooling Layers

The pooling function aggregates information to produce a smaller, **summarized output.**

Although **information is lost** in the pooling layer, it **reduces complexity, improves efficiency**, and **limits the risk of overfitting.**

Fully-connected (FC) layer

Fully-connected (FC) layer: performs the task of classification based on the features extracted through the previous layers and their different filters.

Each neuron in a fully connected layer is connected to every value from the previous layer (just like in a feed-forward NN).

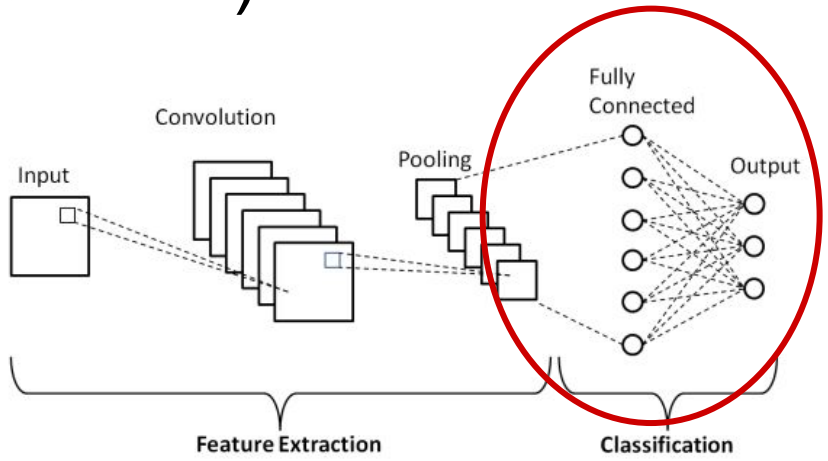


Image Augmentation

Image augmentation refers to generating new training examples after a series of random changes to the training data with the intention of improving model generalization.

The idea is that **by introducing random tweaks** to the training data, the model will **rely less** on certain attributes. For example, we can crop an image in different ways to make the object of interest appear in different positions, thereby reducing the dependence of a model on the position of the object.

Image Augmentation: Flipping

```
# Apply a random horizontal flip 50% of the time
apply(img, torchvision.transforms.RandomHorizontalFlip())
```

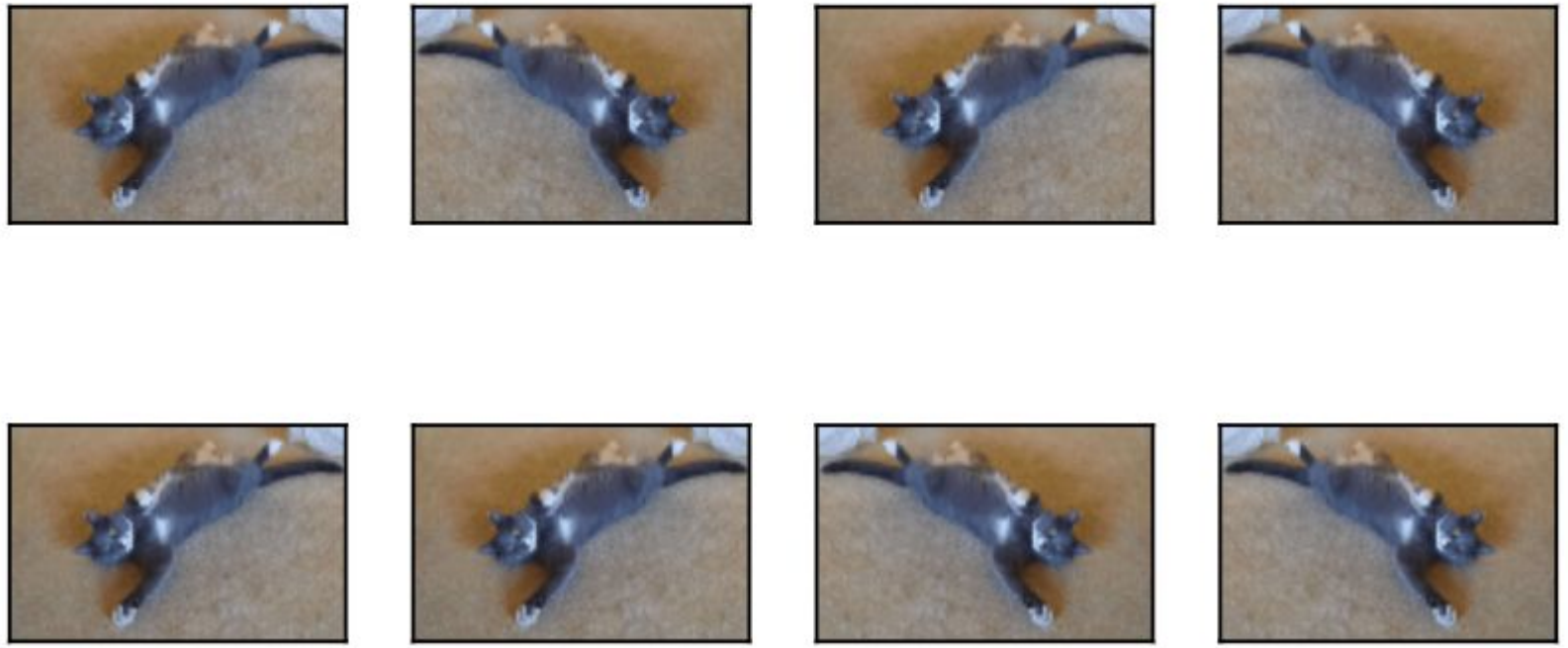


Image Augmentation: Flipping

```
# Apply a random vertical flip 50% of the time
apply(img, torchvision.transforms.RandomVerticalFlip())
```

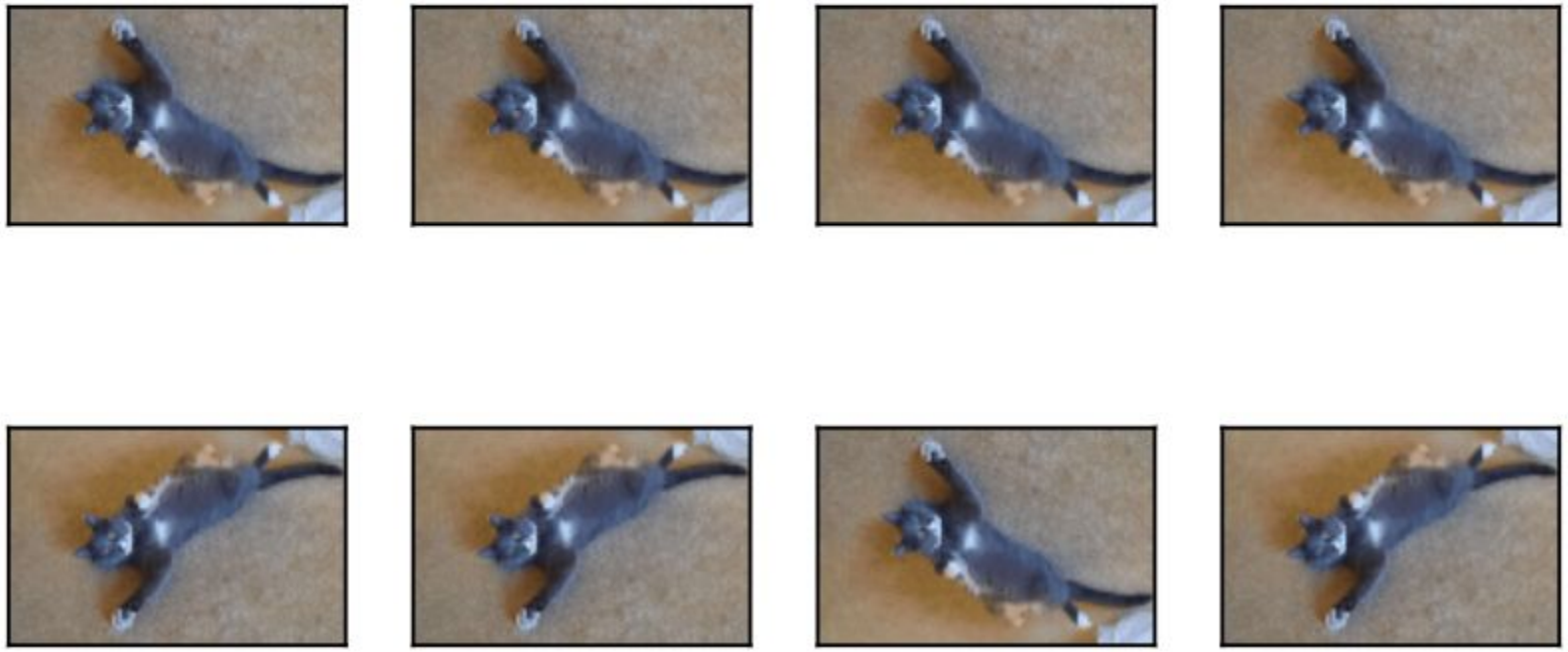


Image Augmentation: Crop

```
# Cropped area will be between 10% and 100% of the original image's area.
# Aspect ratio can vary from tall and narrow (0.5) to wide and short (2).
# Image will be resized to 200 x 200 px
shape_aug = torchvision.transforms.RandomResizedCrop(
    (200, 200), scale=(0.1, 1), ratio=(0.5, 2))
apply(img, shape_aug)
```

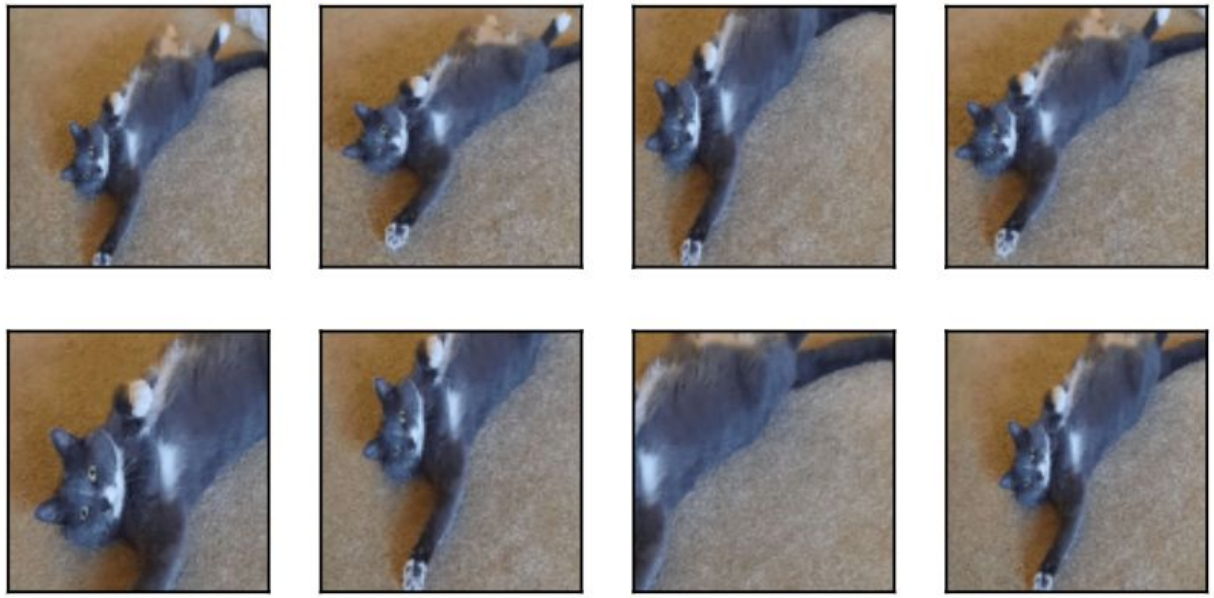


Image Augmentation: Brightness

```
# Randomly change the brightness of the image to a value between 50% and 150%
apply(img, torchvision.transforms.ColorJitter(
    brightness=0.5, contrast=0, saturation=0, hue=0))
```

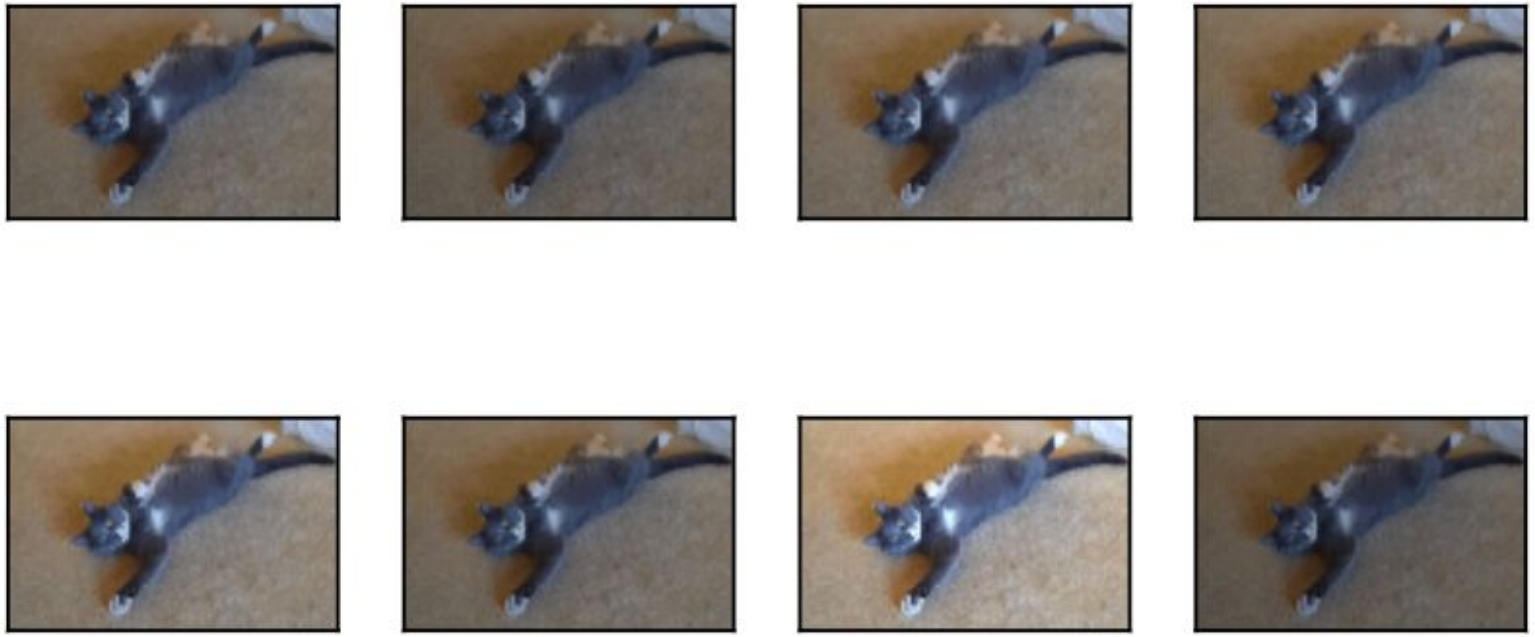


Image Augmentation: Hue

```
# Randomly change the hue
apply(img, torchvision.transforms.ColorJitter(
    brightness=0, contrast=0, saturation=0, hue=0.5))
```

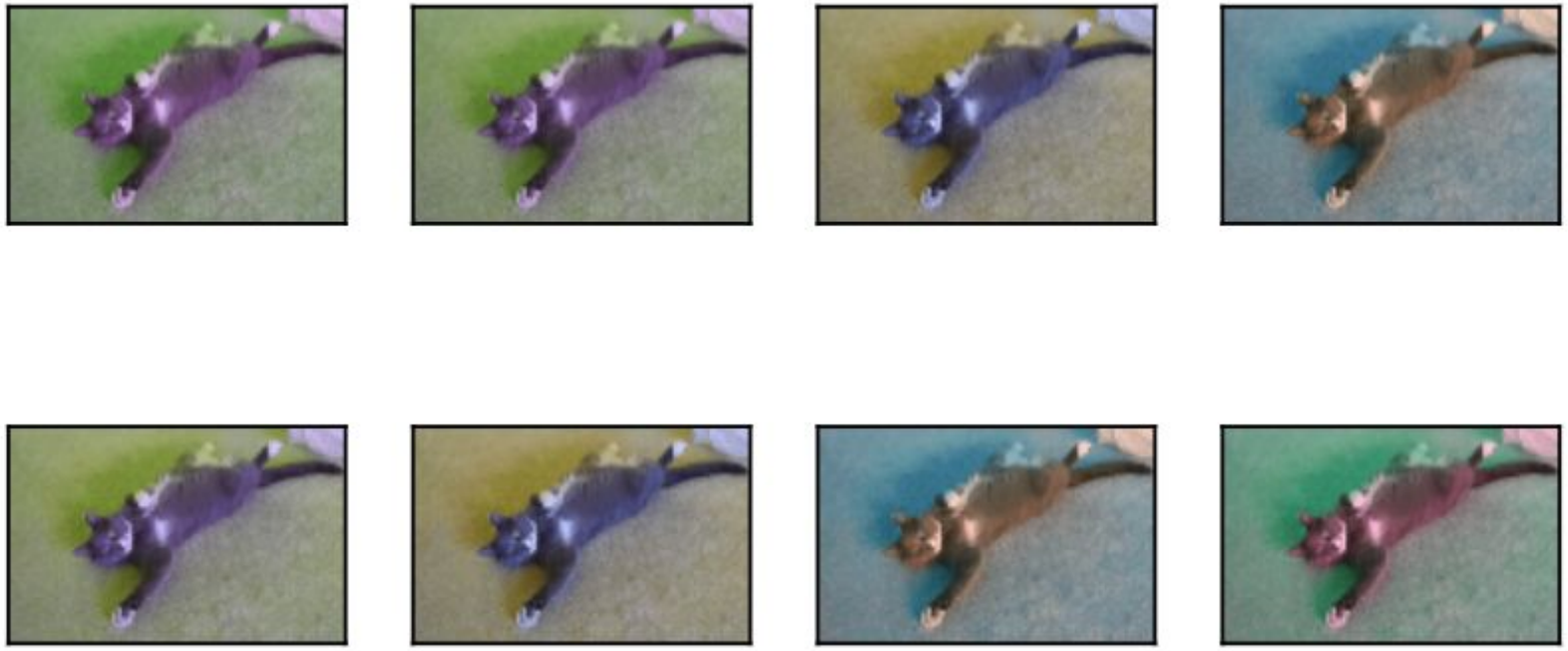


Image Augmentation: Combinations

```
# Randomly change the brightness, contrast, saturation, and hue at the same time
color_aug = torchvision.transforms.ColorJitter(
    brightness=0.5, contrast=0.5, saturation=0.5, hue=0.5)
apply(img, color_aug)
```

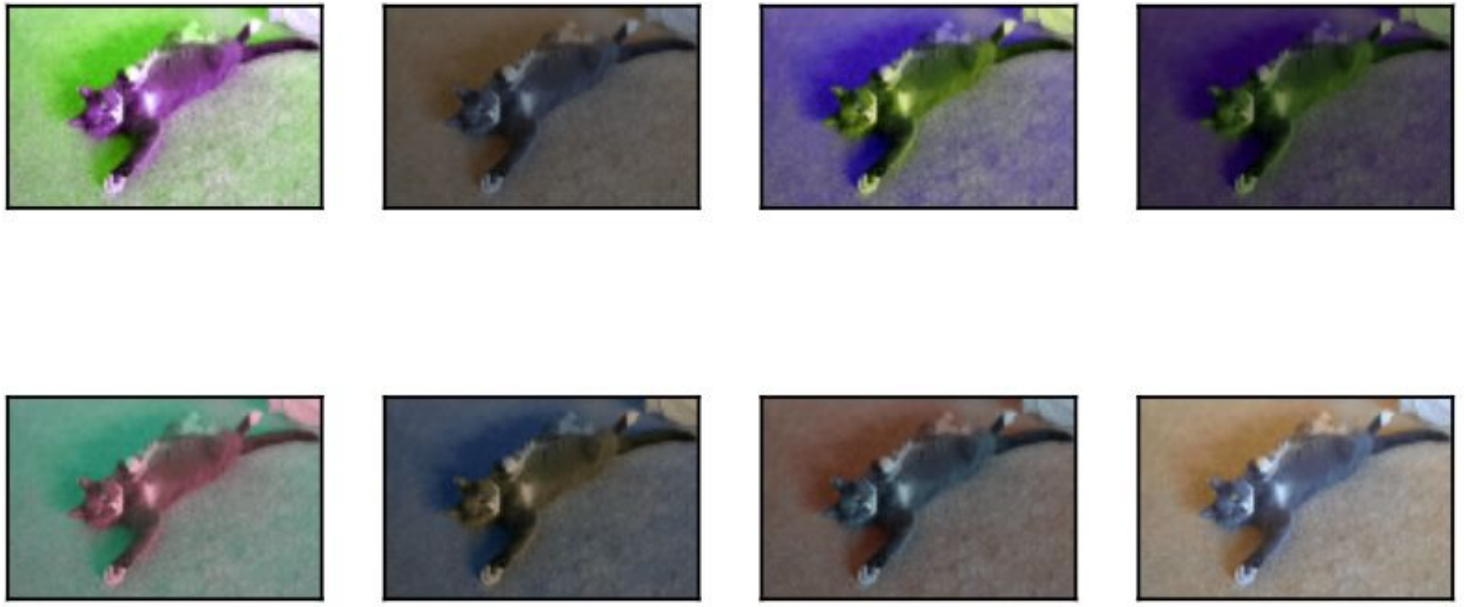
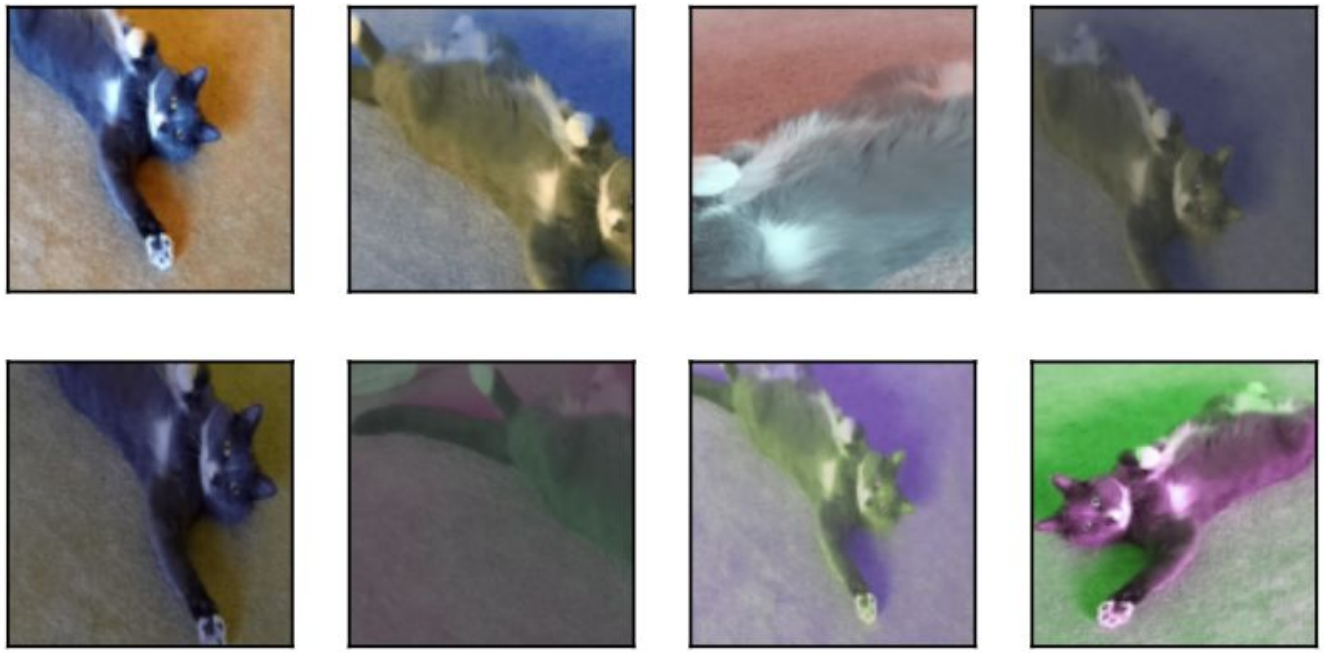


Image Augmentation: Combinations

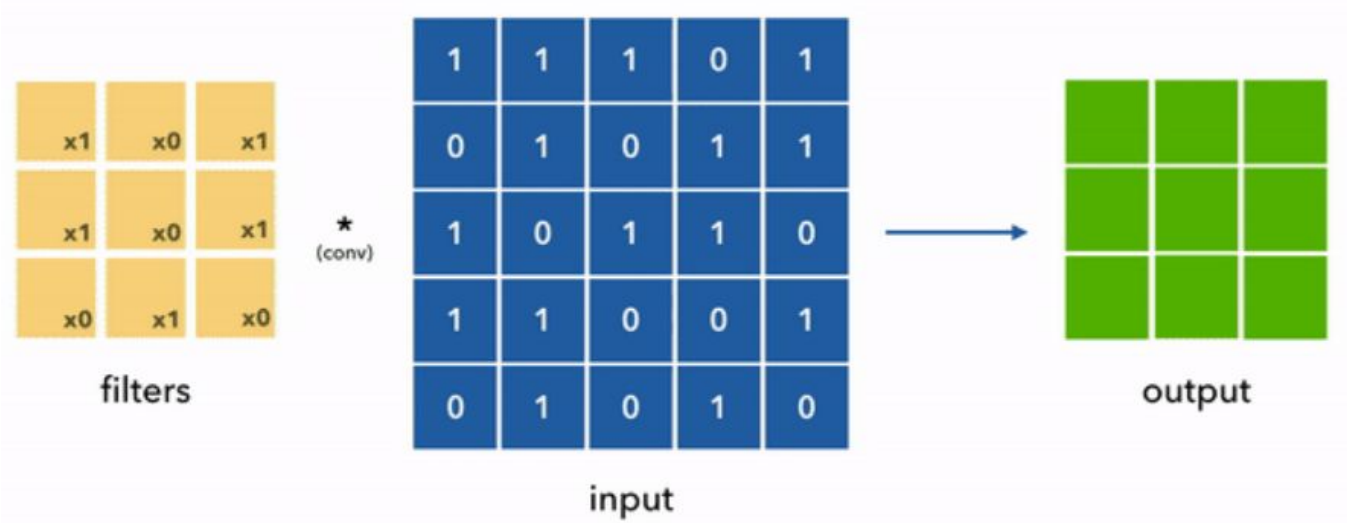
```
# Combining multiple augmentation methods
augs = torchvision.transforms.Compose([
    torchvision.transforms.RandomHorizontalFlip(), color_aug, shape_aug])
apply(img, augs)
```



Padding

One issue when applying **convolutional layers** is that we **lose pixels** on the perimeter of our image.

This can add up when we have multiple convolutional layers.

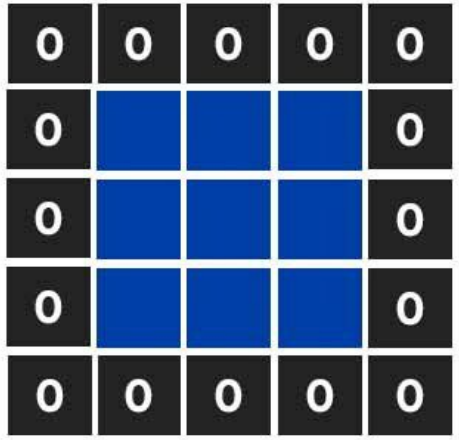


Padding

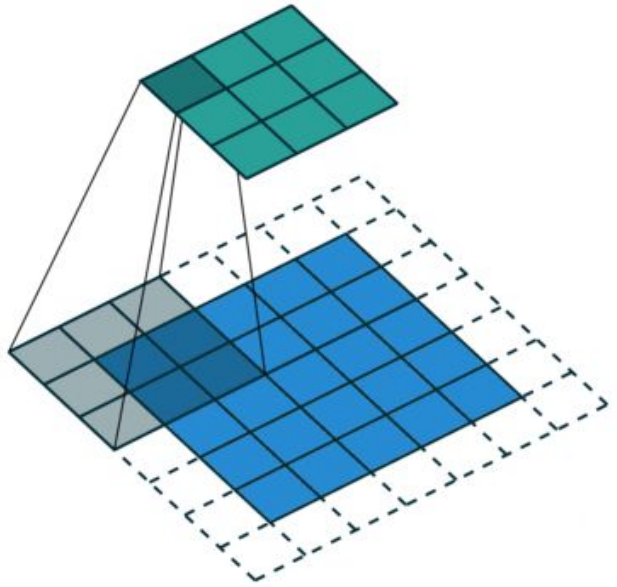
One solution to this is to **add extra pixels of filler** around the boundary of our **input image**. This **increases the size** of the image. Typically, we set the values of the extra pixels to zero.

Input Image

Appyling padding of 1 on 3X3



Padded Image



Activation Functions

Between each layer, we need to introduce **non-linearity.** Typically, these activation functions are used:

Convolutional Layers → ReLU

Pooling Layers → ReLU

FC Layer → Softmax

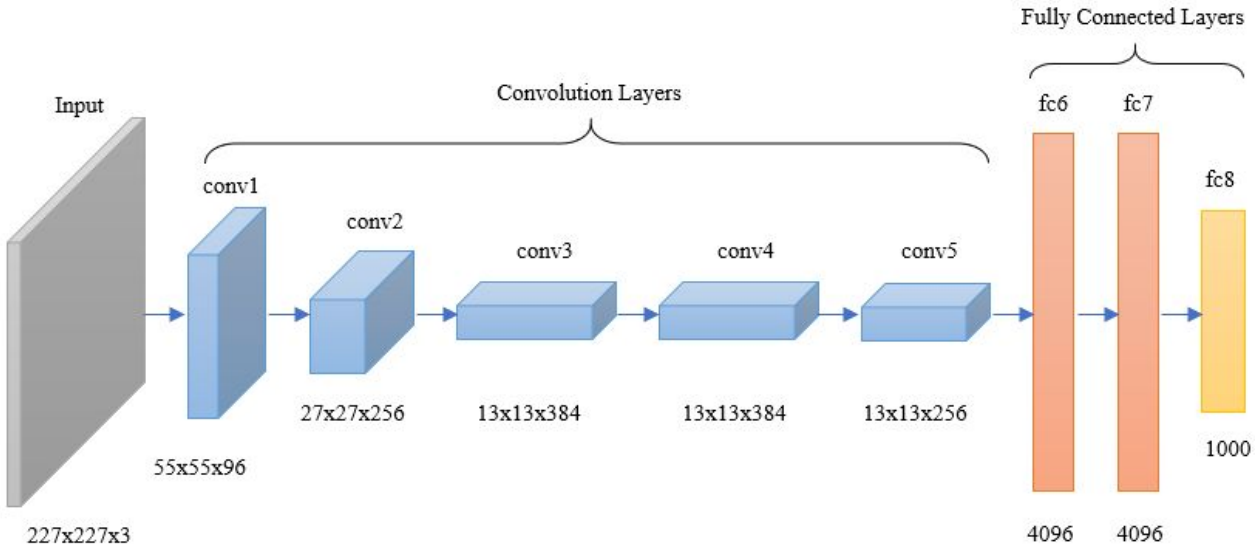
Training CNNs

```python
class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 6, 5)    # First convolutional layer: 3 input channels (RGB), 6 output filters, 5x5 kernel
        self.pool = nn.MaxPool2d(2, 2)     # Max pooling layer: reduces spatial size by taking 2x2 regions with stride 2
        self.conv2 = nn.Conv2d(6, 16, 5)   # Second convolutional layer: 6 input channels, 16 output filters, 5x5 kernel
        self.fc1 = nn.Linear(16 * 5 * 5, 120)  # First fully connected (dense) layer: 16*5*5 inputs - 120 outputs
        self.fc2 = nn.Linear(120, 84)      # Second fully connected layer: 120 inputs - 84 outputs
        self.fc3 = nn.Linear(84, 10)       # Output layer: 84 inputs - 10 outputs (for 10 classes, e.g., CIFAR-10)
```

AlexNet

In 2012, a CNN called **AlexNet** won the ImageNet challenge. The challenge was to correctly classify images and detect different objects and scenes across 1000 categories. Every year, the team that produced the model with the lowest error rate won. This network showed, for the first time, that the **features obtained by learning** can outperform manually-designed features.

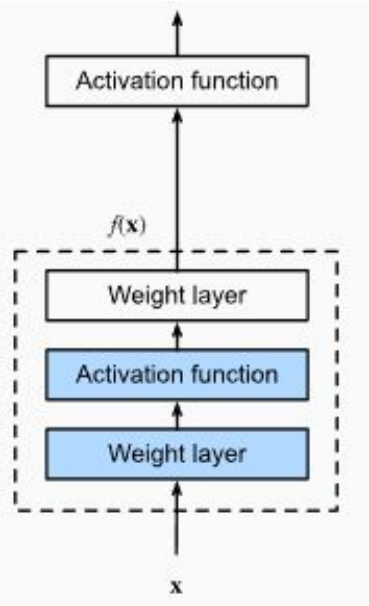
AlexNet has 8 layers: 5 convolutional layers followed by 3 fully-connected linear layers.



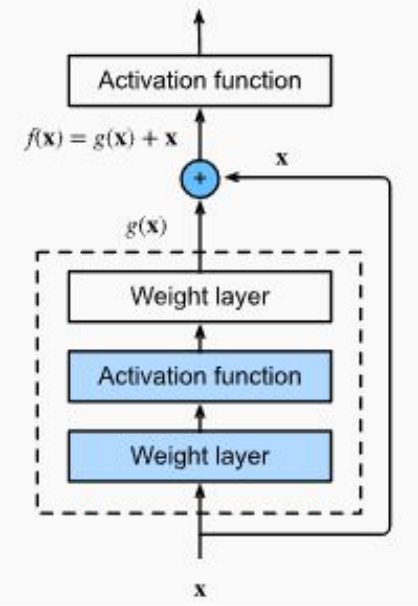
ResNet

In 2015, a CNN called ResNet won the ImageNet competition. It introduced something called **residual blocks** to the architecture. These **residual blocks** let the network learn **only the difference (residual)** between the input and the desired output.

Regular Block



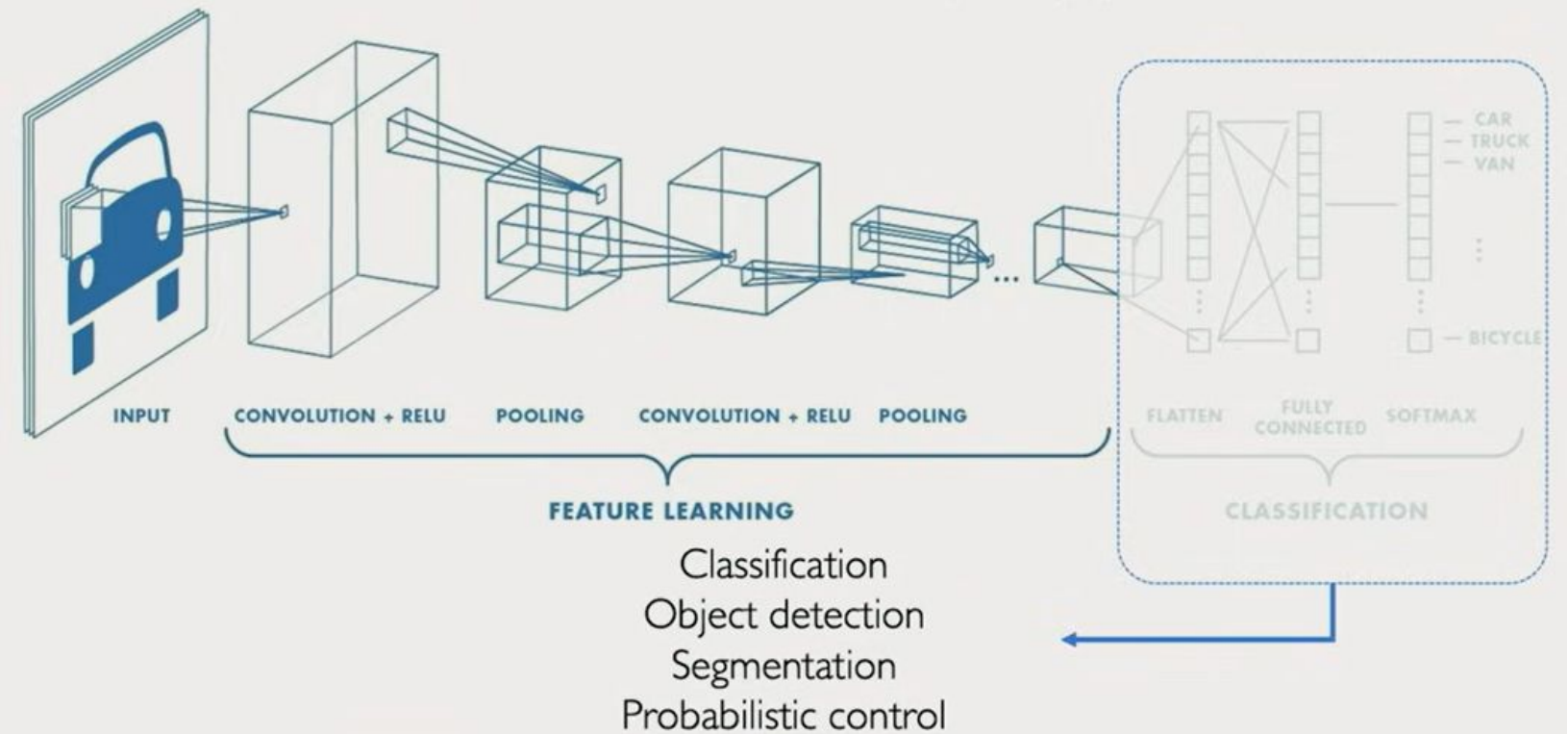
Residual Block



PyTorch has 5 versions of ResNet available for use with 18, 34, 50, 101, 152 layers.

CNNs are Flexible

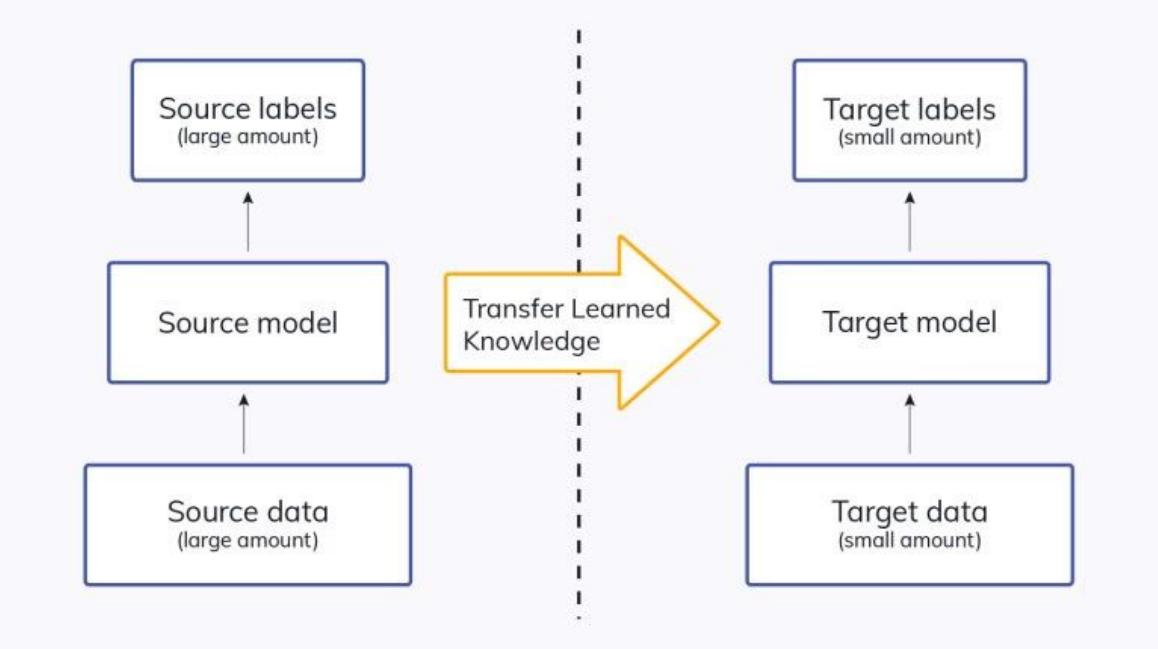
We've mostly focused on classification, but CNNs have many applications! This is because they can be split into 2 main parts:



Transfer Learning

CNNs require large, hand-labelled datasets to be trained on. These are costly to obtain, so instead, we can use **transfer learning.**

Transfer learning is the process of transferring the knowledge learned from the *source dataset* to the *target dataset.*

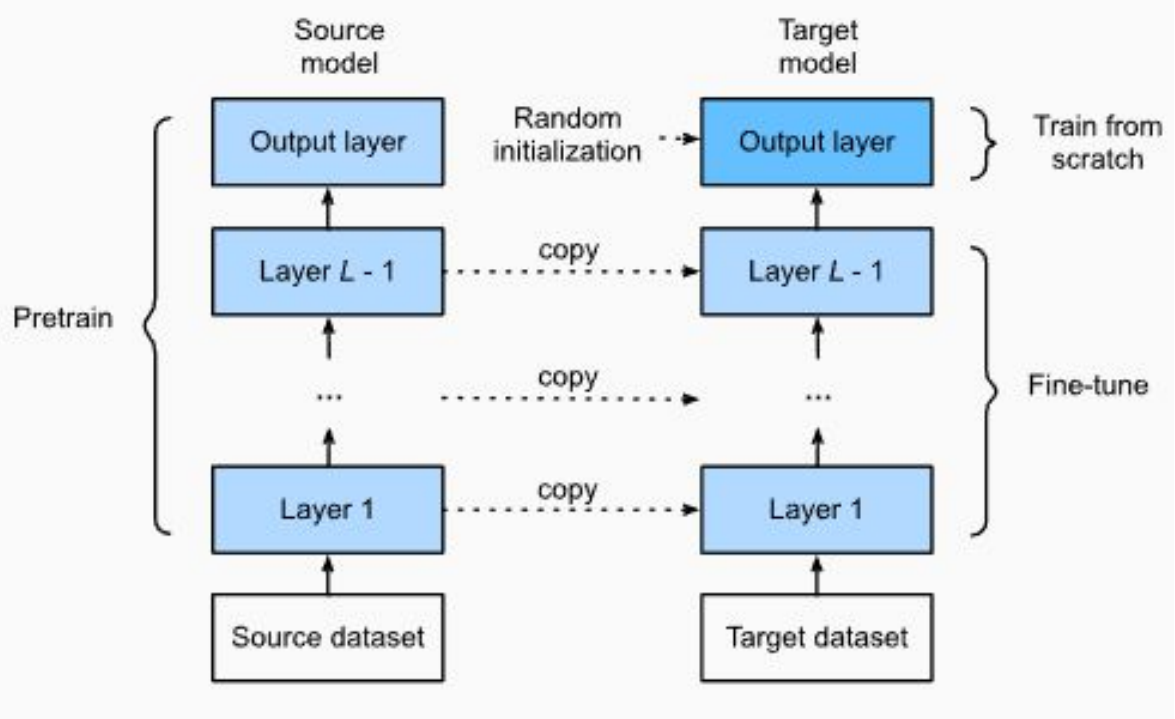


For example, knowledge gained while learning to recognize cars could be applied when trying to recognize trucks.

Fine-Tuning

Fine-tuning is a common technique in transfer learning.

- The **target model** copies all model designs with their parameters from the source model except the output layer.
- It fine-tunes these parameters based on the target dataset **(small learning rate)**.
- The output layer of the target model needs to be trained from scratch **(larger learning rate)**.



Summary

- **Computer vision** is an area of AI concerned with learning from visual data. CNNs are a type neural network used in computer vision.
- CNNs utilize the **spatial structure** in data to make predictions. They also typically have less parameters than fully connected networks.
- CNNs use **filters/kernels** which **"convolve"** over the input.
- The main layers of CNNs are: **convolutional layers, pooling layers, fully connected layers.**
- **Image augmentations** can help CNNs generalize better.

Resources

Interact with Different Kernels

List of Transformations Supported by Torchvision

AlexNet History and Tutorial

Nice Tutorial on CNNs

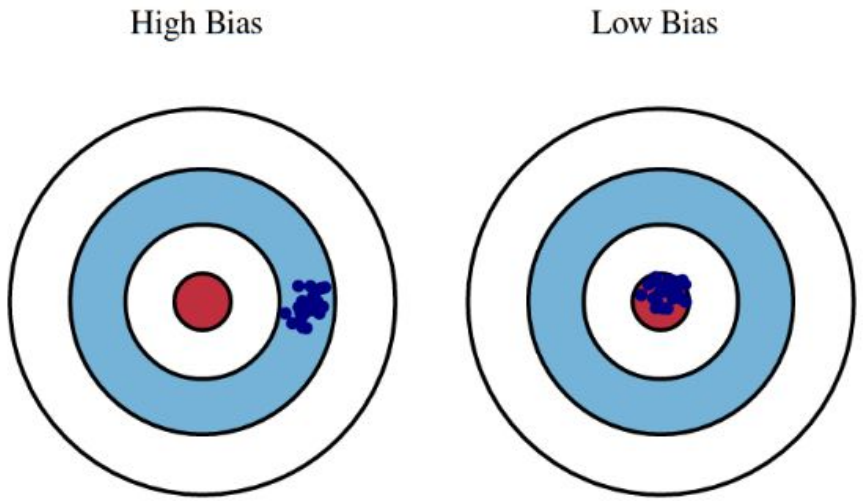
Bias and Fairness in Machine Learning

Bias (Statistical Definition)

In an earlier lecture, we defined **statistical** bias as the **expected difference** between an estimator and the true parameter.

If bias = 0 → the estimator is unbiased

If bias is high → the estimates are systematically off target.



Bias

In **machine learning**, bias can also refer to systematic errors introduced by the data, model, or human decisions.

- The training data is not representative of the population (e.g. some groups are under-represented).
- The training data is poorly or incorrectly labelled.
- The data we use is historically biased (e.g. models assigning roles and characteristics based on traditional gender norms).

This kind of bias is undesirable because a model trained on biased data will **perform poorly on underrepresented groups**, leading to unreliable predictions. It will also **not generalize well** when deployed in the real world.

Bias in Image Data

Oftentimes for image classification tasks, **crowdsourcing** is used to hand-label data. This poses several risks:

- **Inconsistent judgments** - e.g. an "angry" expression can be interpreted differently.
- **Lack of domain knowledge / expertise** - e.g. "dog" vs. "german shepherd".
- **Economic and motivational factors** - Crowdsourcing platforms often pay per task, leading workers to *prioritize speed over accuracy*.

Fairness

Machine learning (ML) models are not inherently objective. **Fairness** is about ensuring models **generalize equitably.**

- We saw how if data is biased, models may unfairly favor or disadvantage certain groups.

This can be harmful in cases where sensitive (demographic) data is being considered. For example, loan / credit approvals, university admissions, insurance, etc.

Mitigating Bias, Improving Fairness

There are several ways to mitigate bias and improve model fairness:

- Whenever possible, use **representative training data** (your model is only as good as the data you feed it).
  - This could mean collecting more data, balancing the dataset, etc.
- Evaluate **model performance** across subgroups.
- Adjust the **training process.**
  - Reweighting, resampling, and/or regularization.
- Post-processing
  - After training, adjust **decision thresholds** to equalize metrics across groups.

Resources

Fairness in ML Example

Types of Bias

Sequential Neural Networks

Outline

- Sequential Data
- Sequential Models
- RNNs
  - LSTMs
- Transformers

Sequential Data

In earlier lectures, we focused on models that can handle **fixed-length data** such as regression models, trees, and feed-forward neural networks.

In this lecture, we saw that CNNs work with image data, which is also considered **fixed-length data** but handles hierarchical structure and invariances.

What about **sequential data** where the **order matters** and the inputs may have **variable length?**

Sequential Data

Sequential data refers to data in which the **order matters.** There are many examples of sequential data:

- **Text -** word order determines meaning.
- **Audio / Speech -** each sample depends on previous vibrations.
- **Time Series Data -** measurements taken at regular intervals over time, like stock prices, temperature readings, and sensor data.

Sequential Models

There are several types of models which can handle **sequential data.** Today we will cover **3 neural network architectures** which can be used with sequential data:

1. **Recurrent Neural Networks (RNNs)**
2. **Long Short-Term Memory Networks (LSTMs)**
3. **Transformers**

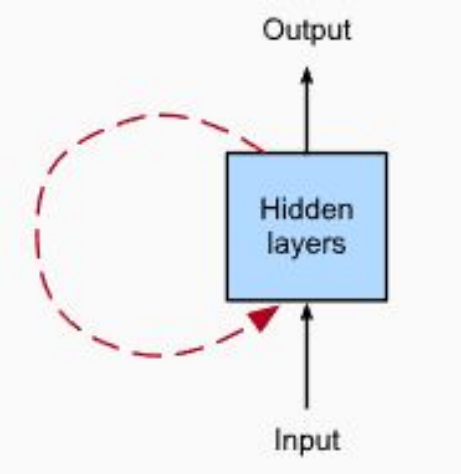
Sequential Models

If we want a neural network to process and learn from sequential data, it should be able to:

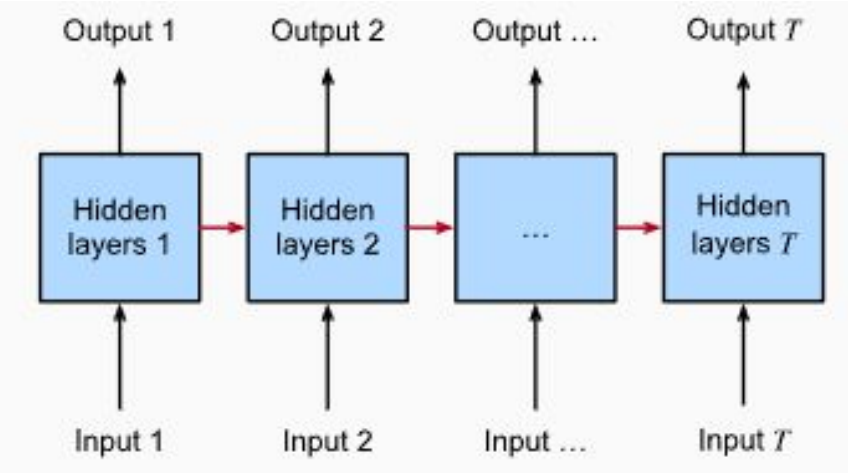
- Capture both **short-term and long-term dependencies** that exist across the sequence.
- Maintain information about the order of the elements.
- Share parameters across sequence positions.
- *Handle variable length sequences (not required, but helpful).*

Recurrent Neural Networks (RNNs)

Recurrent neural networks (RNNs) are deep learning models that can process sequential data using **recurrent connections**. Recurrent neural networks are **unrolled** across sequence steps, with the **same underlying parameters** applied at each step.



Rolled



Unrolled

Recurrent Neural Networks (RNNs)

RNNs track context by maintaining a **hidden state** at each time step. This **hidden state** acts as a **memory** that stores information about previous inputs.

A **feedback loop** is created by passing the hidden state from one-time step to the next.

At each time step, the RNN processes the current input and the hidden state from the previous time step. This allows the RNN to "remember" previous data points and use that information to influence the current output.

RNNs

Pros:

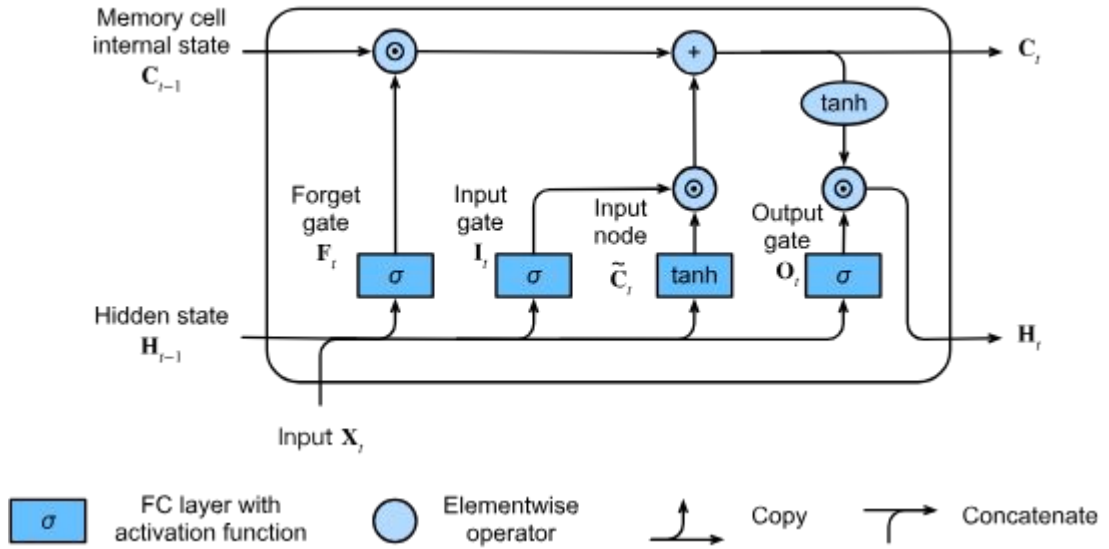
- Good for sequential data.
- Simple architecture.

Cons:

- Limited memory - is only able to capture short-term dependencies well.
- Suffers from exploding and vanishing gradients which can make training difficult.

Long Short-Term Memory Networks (LSTMs)

Long Short-Term Memory Networks (LSTMs) are type of RNN that introduced a **memory cell.** By providing a separate path for short and long-term memory, LSTMs can remember information for longer and are less susceptible to the vanishing gradient problem of RNNs.



Memory Cells

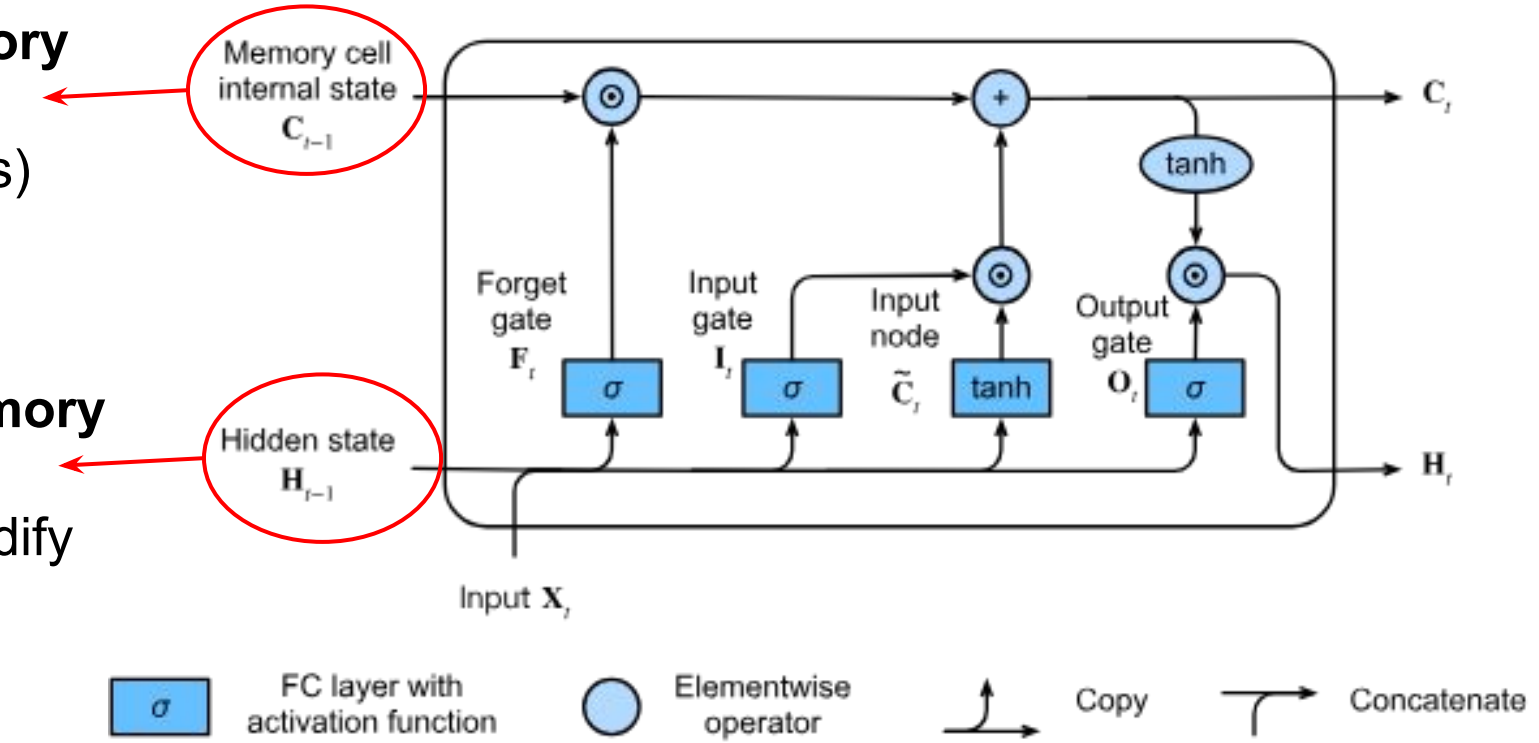
Memory cells have 3 **gates** that control the flow of information**:**

1. **Input gate -** determines how much of the input node's value should be added to the current memory cell internal state.
2. **Forget gate -** determines whether to keep the current value of the memory or flush it.
3. **Output gate -** determines whether the memory cell should influence the output at the current time step.

Long Short-Term Memory Networks (LSTMs)

Keeps track of **long-term memory** (not modified by weights or biases)

Keeps track of **short-term memory** (connected to weights that modify them)



LSTMs

Pros:

- Handles long-term dependencies.
- Reduces the exploding / vanishing gradient problem.

Cons:

- Complex architecture means slower training and more parameters to train.
- Cannot be parallelized easily.

Transformers

Transformers are a type of neural network that excel at processing sequential data. Their key feature is an **attention mechanism.**

Unlike RNNs and LSTMs which are **serialized** (process one input at a time, in order), the **attention mechanism** allows transformers to process all positions of the **entire sequence simultaneously.**

Transformers

Transformers *(mostly*)* have an **encoder-decoder architecture.** At a high-level, they are made up of these 2 components:

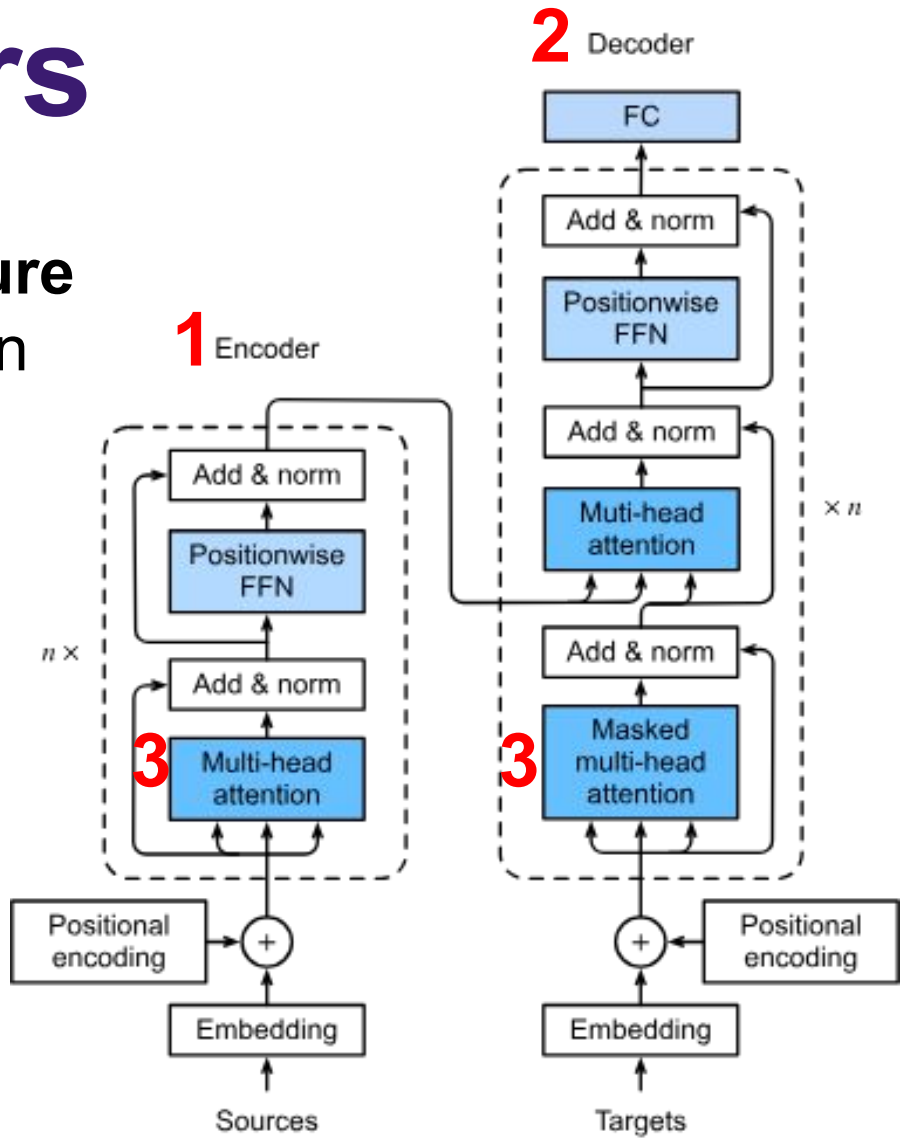
- The **encoder** is a neural network that converts input into an intermediate representation.
- A **decoder** is another neural network that converts that intermediate representation into a useful output.

** Encoder-only transformers (e.g. BERT) also exist, but are beyond the scope of this lecture.*

Transformers

A **transformer architecture** diagram. There are 3 main components:

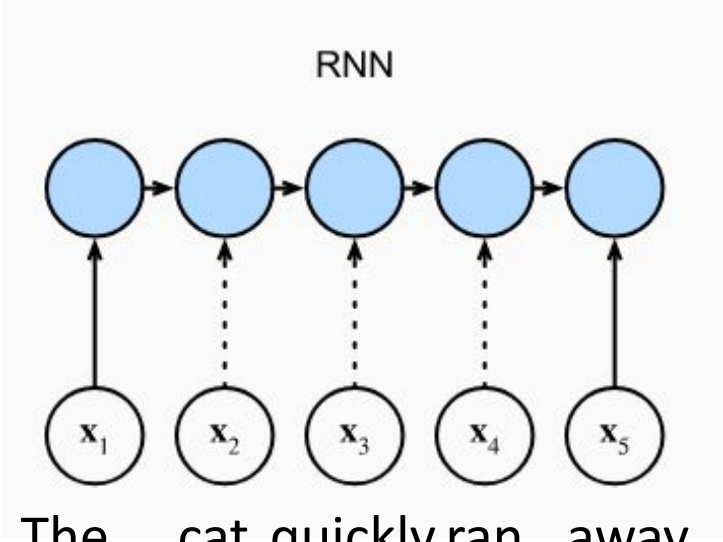
1. Encoder
2. Decoder
3. Attention

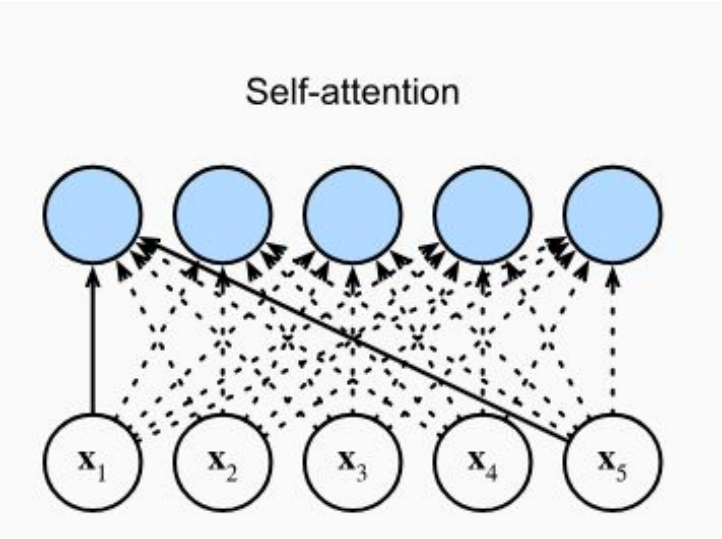


Attention

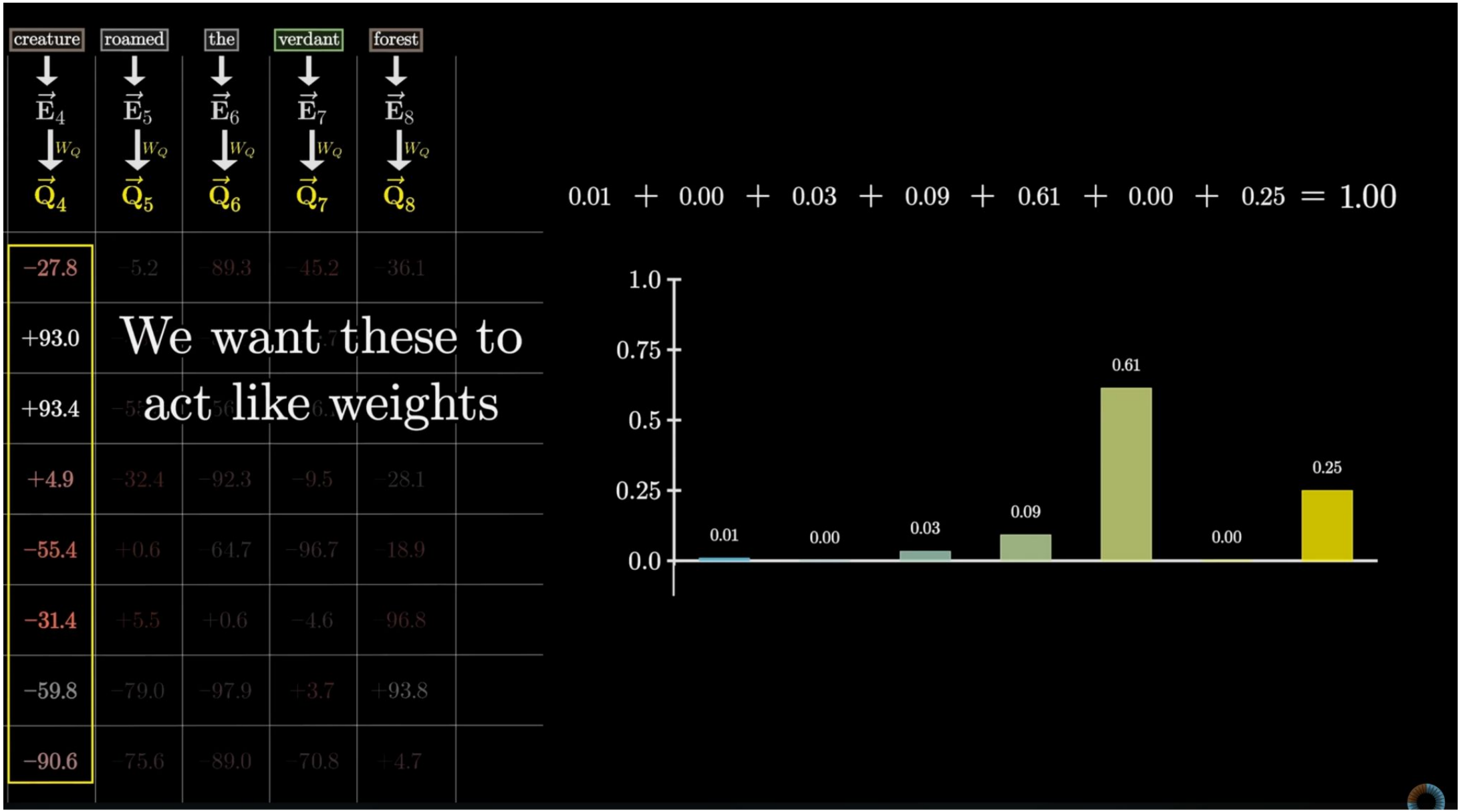
Attention mechanisms are algorithms designed to determine which parts of a data sequence a model should "pay attention to" at any particular moment.

- Transformers have an additional path for each input value so that at each step, the decoder can access those values.

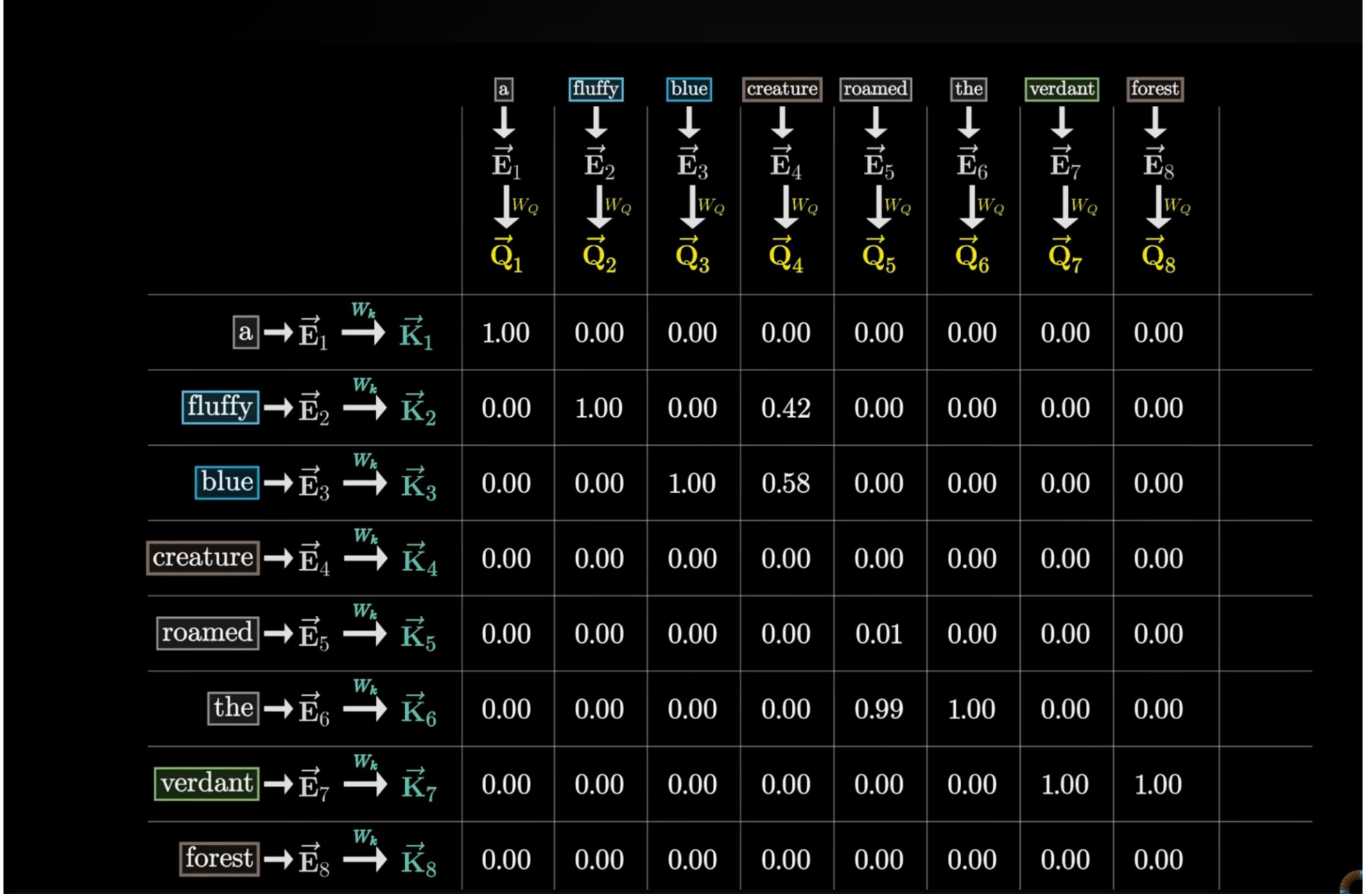




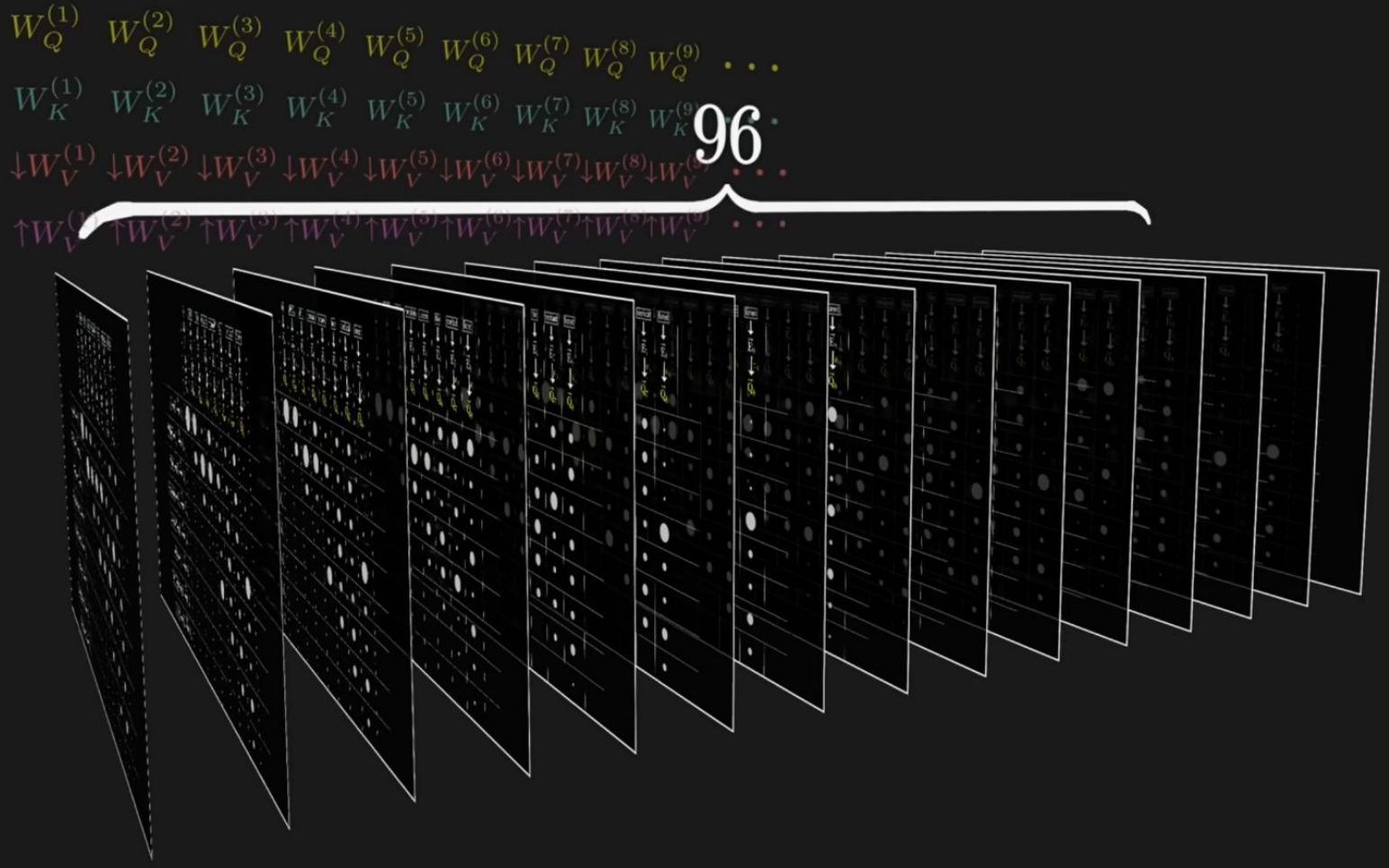
	a	fluffy	blue	creature	roamed	the	verdant	forest
	$\vec{E}_1$	$\vec{E}_2$	$\vec{E}_3$	$\vec{E}_4$	$\vec{E}_5$	$\vec{E}_6$	$\vec{E}_7$	$\vec{E}_8$
	$\downarrow W_Q$	$\downarrow W_Q$	$\downarrow W_Q$	$\downarrow W_Q$	$\downarrow W_Q$	$\downarrow W_Q$	$\downarrow W_Q$	$\downarrow W_Q$
	$\vec{Q}_1$	$\vec{Q}_2$	$\vec{Q}_3$	$\vec{Q}_4$	$\vec{Q}_5$	$\vec{Q}_6$	$\vec{Q}_7$	$\vec{Q}_8$
a $\rightarrow \vec{E}_1 \xrightarrow{W_K} \vec{K}_1$	+0.7	−83.7	−24.7	−27.8	−5.2	−89.3	−45.2	−36.1
fluffy $\rightarrow \vec{E}_2 \xrightarrow{W_K} \vec{K}_2$	−73.4	+2.9	−5.4	+93.0	−48.2	−87.3	−49.7	+7.8
blue $\rightarrow \vec{E}_3 \xrightarrow{W_K} \vec{K}_3$	−53.4	−5.7	+1.8	+93.4	−55.6	−56.0	−26.1	−62.1
creature $\rightarrow \vec{E}_4 \xrightarrow{W_K} \vec{K}_4$	−21.5	−29.7	−56.1	+4.9	−32.4	−92.3	−9.5	−28.1
roamed $\rightarrow \vec{E}_5 \xrightarrow{W_K} \vec{K}_5$	−20.1	−40.9	−87.8	−55.4	+0.6	−64.7	−96.7	−18.9
the $\rightarrow \vec{E}_6 \xrightarrow{W_K} \vec{K}_6$	−87.9	−33.3	−22.6	−31.4	+5.5	+0.6	−4.6	−96.8
verdant $\rightarrow \vec{E}_7 \xrightarrow{W_K} \vec{K}_7$	−41.2	−55.5	−42.3	−59.8	−79.0	−97.9	+3.7	+93.8
forest $\rightarrow \vec{E}_8 \xrightarrow{W_K} \vec{K}_8$	−58.9	−75.5	−91.1	−90.6	−75.6	−89.0	−70.8	+4.7


creature
roamed
the
verdant
forest
$\vec{E}_4$
$\vec{E}_5$
$\vec{E}_6$
$\vec{E}_7$
$\vec{E}_8$
W_Q
$\vec{Q}_4$
$\vec{Q}_5$
$\vec{Q}_6$
$\vec{Q}_7$
$\vec{Q}_8$
−27.8 −5.2 89.3 −45.2 −36.1
+93.0
+93.4
+4.9 −32.4 −92.3 −9.5 −28.1
−55.4 +0.6 −64.7 −96.7 −18.9
−31.4 +5.5 +0.6 −4.6 −96.8
−59.8 −79.0 −97.9 +3.7 +93.8
−90.6 −75.6 −89.0 −70.8 +4.7
We want these to act like weights
0.01 + 0.00 + 0.03 + 0.09 + 0.61 + 0.00 + 0.25 = 1.00
1.0
0.75
0.5
0.25
0.0
0.01
0.00
0.03
0.09
0.61
0.00
0.25


a
fluffy
blue
creature
roamed
the
verdant
forest
$\vec{E}_1$ W_Q $\vec{Q}_1$
$\vec{E}_2$ W_Q $\vec{Q}_2$
$\vec{E}_3$ W_Q $\vec{Q}_3$
$\vec{E}_4$ W_Q $\vec{Q}_4$
$\vec{E}_5$ W_Q $\vec{Q}_5$
$\vec{E}_6$ W_Q $\vec{Q}_6$
$\vec{E}_7$ W_Q $\vec{Q}_7$
$\vec{E}_8$ W_Q $\vec{Q}_8$
a → $\vec{E}_1$ W_K → $\vec{K}_1$
fluffy → $\vec{E}_2$ W_K → $\vec{K}_2$
blue → $\vec{E}_3$ W_K → $\vec{K}_3$
creature → $\vec{E}_4$ W_K → $\vec{K}_4$
roamed → $\vec{E}_5$ W_K → $\vec{K}_5$
the → $\vec{E}_6$ W_K → $\vec{K}_6$
verdant → $\vec{E}_7$ W_K → $\vec{K}_7$
forest → $\vec{E}_8$ W_K → $\vec{K}_8$
+0.7
−73.4
−53.4
−21.5
−20.1
−87.9
−41.2
−58.9
−24.7
5.4
+1.8
56.1
−87.8
−22.6
−42.3
91.1
−27.8
+93.0
+93.4
+4.9
−55.4
−31.4
−59.8
−90.6
softmax
0.00
0.42
0.58
0.00
0.00
0.00
0.00
0.00


a
fluffy
blue
creature
roamed
the
verdant
forest
$\vec{E}_1$ $\vec{E}_2$ $\vec{E}_3$ $\vec{E}_4$ $\vec{E}_5$ $\vec{E}_6$ $\vec{E}_7$ $\vec{E}_8$
W_Q
$\vec{Q}_1$ $\vec{Q}_2$ $\vec{Q}_3$ $\vec{Q}_4$ $\vec{Q}_5$ $\vec{Q}_6$ $\vec{Q}_7$ $\vec{Q}_8$
a → $\vec{E}_1$ W_k $\vec{K}_1$ 1.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
fluffy → $\vec{E}_2$ W_k $\vec{K}_2$ 0.00 1.00 0.00 0.42 0.00 0.00 0.00 0.00
blue → $\vec{E}_3$ W_k $\vec{K}_3$ 0.00 0.00 1.00 0.58 0.00 0.00 0.00 0.00
creature → $\vec{E}_4$ W_k $\vec{K}_4$ 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
roamed → $\vec{E}_5$ W_k $\vec{K}_5$ 0.00 0.00 0.00 0.00 0.01 0.00 0.00 0.00
the → $\vec{E}_6$ W_k $\vec{K}_6$ 0.00 0.00 0.00 0.00 0.99 1.00 0.00 0.00
verdant → $\vec{E}_7$ W_k $\vec{K}_7$ 0.00 0.00 0.00 0.00 0.00 0.00 1.00 1.00
forest → $\vec{E}_8$ W_k $\vec{K}_8$ 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00

Multi-headed attention



For More on Transformer:

Transformers Part 1

Transformers Part 2

Transformers

Pros:

- Captures long-range dependencies easily.
- Parallelizable (faster training).
- Outperforms other sequential models.
- Applicable to many types of ML problems.

Cons:

- Computationally expensive.
- Requires large datasets.

Summary

- Just like how CNNs work with **spatial structure,** sequential models work with **temporal structure.**
- **Recurrent Neural Networks** use a **hidden state** to act as "memory", however they suffer from exploding / vanishing gradients, and don't capture long-term dependencies well.
- **LSTMs** are a type of RNN that use **memory cells** and are better at maintaining long and short term memories.
- **Transformers** are an extremely popular kind of NN that **outperform** both RNNs and LSTMs. They excel at processing sequential data due to their **attention mechanism.**